



國立成功大學

National Cheng Kung University

Introduction to Emerging Memory Techniques

Lecture 6: Processing in Memory

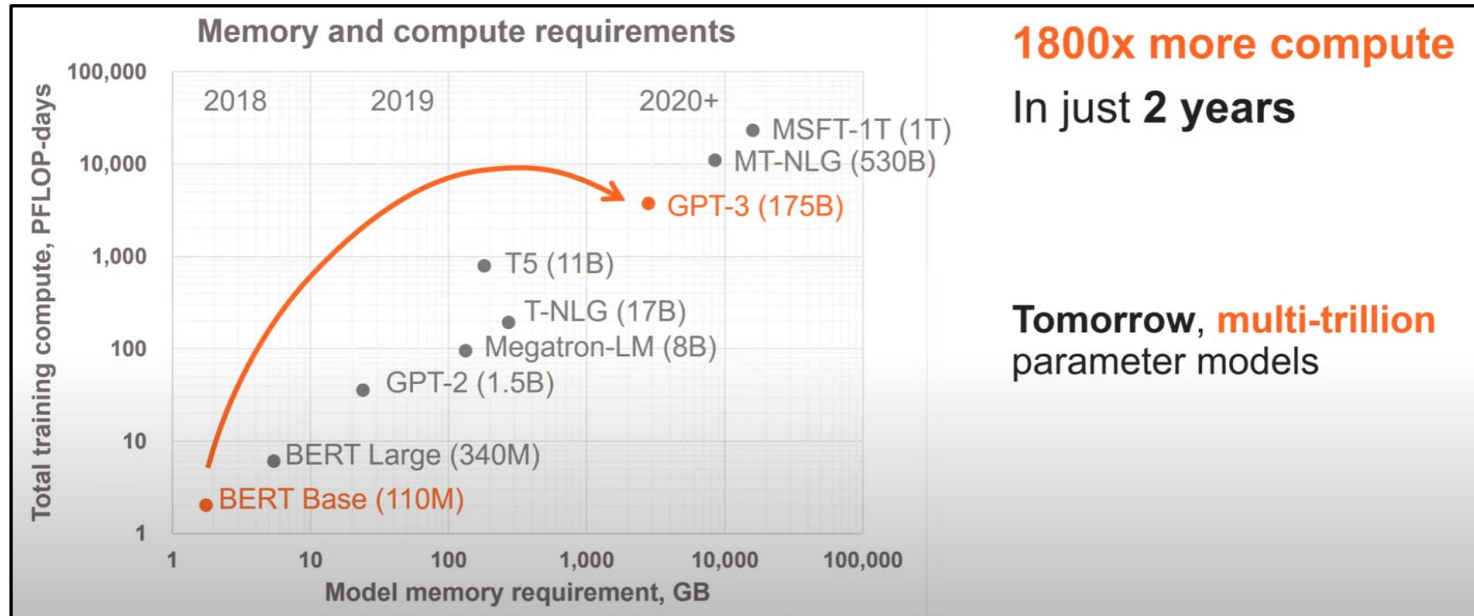
Prof. Chien-Chung Ho

Computing is Bottlenecked by Data

- Data is key for AI, ML, genomics,
- Important workloads are all data intensive
 - Generic sequencing, financial analysis, social media and big data analytics, image recognition and classification, speech recognition and NLP, Training DL models,
- They require rapid and efficient processing of large amounts of data
- Data is increasing, since
 - ✓– We can generate more than we can process
 - ✓– We need to perform more sophisticated analyses on more data

Huge Requirements for Performance & Efficiency

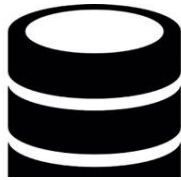
- Exponential growth of neural networks



GPT-4 was released in 2023, and it has **1.76T**(1.76×10^{12}) parameters
10x more compute and memory demand are generated!!

This is not only happened on NN!

- Data gradually becomes, it can be said it dominates the key for **MODERN** and **FUTURE** workloads
 - Computer-based environments



Data overwhelms modern machines nowadays!

Data further results in serious performance and energy bottlenecks

– **MOBILE DEVICES**



Web browser



Video playback/capture
Video codec



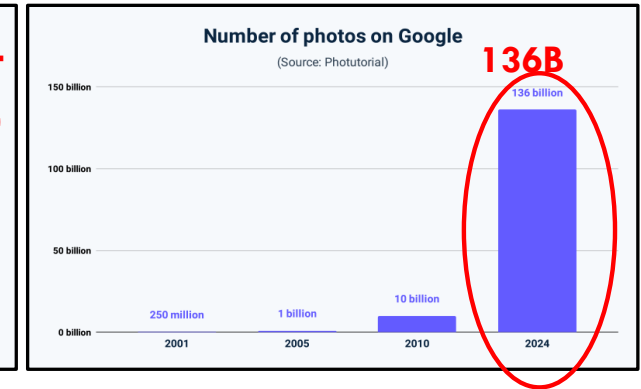
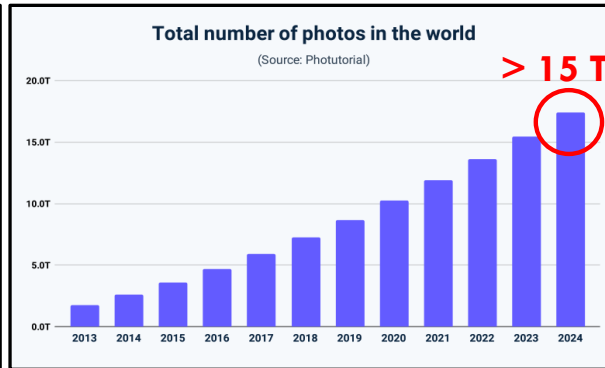
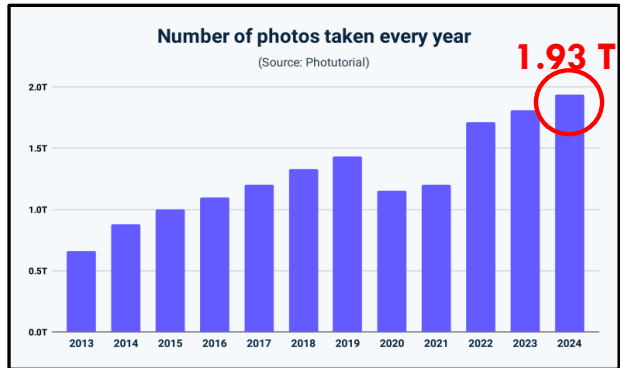
TensorFlow Mobile
ML framework

The Rise of New Technologies

Driving Data Explosion – Image Data

- There has been an explosive generation of image data.

Source: <https://photutorial.com/photos-statistics/>



- 89% of photos were with smartphones in 2020, **92.5% in 2023**, and 93% in 2023.
 - The widespread adoption of smartphones has driven the explosive growth of image data.



1990~2000



2000~2010



2010~2011

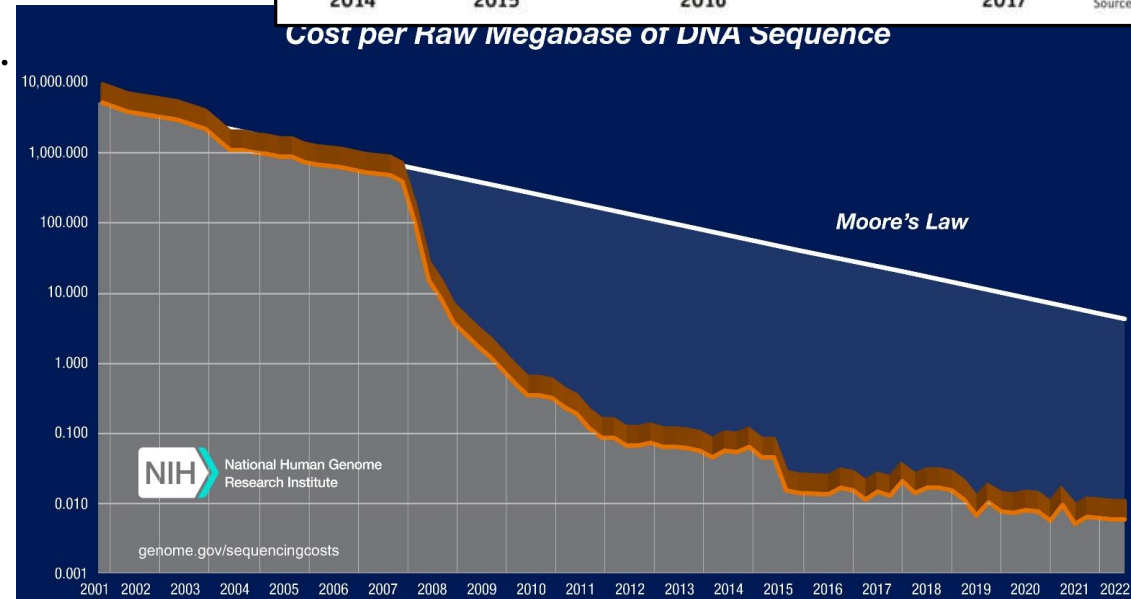
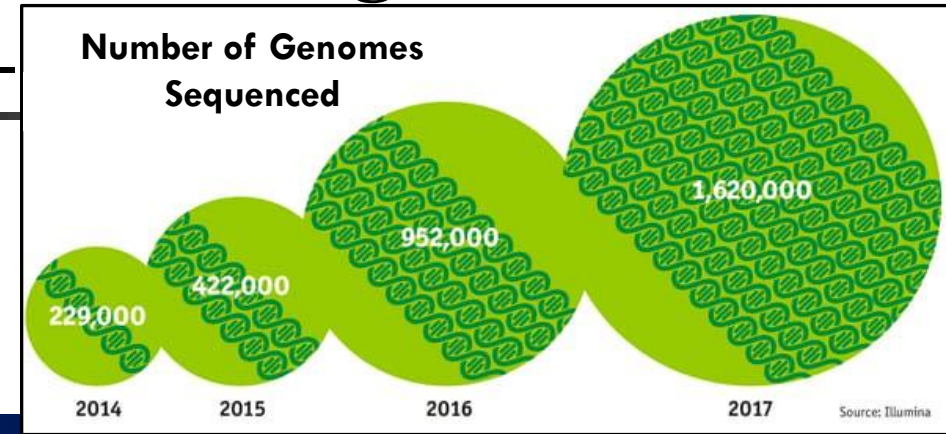


2011~now

The Rise of New Technologies

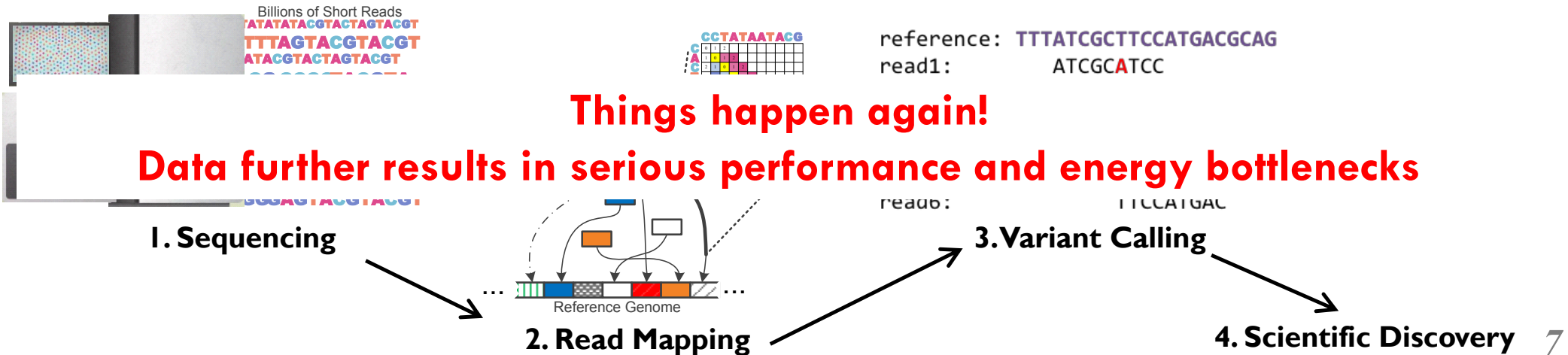
Driving Data Explosion –

- Genome analysis can be helpful in various applications, including:
 - Diagnosis and treatment of genetic diseases, drug development, agriculture, etc.
- In 2014 it became possible to sequence human genomes, at volume, for \$1,000 a time.
- The cost of DNA sequencing continues to decline.



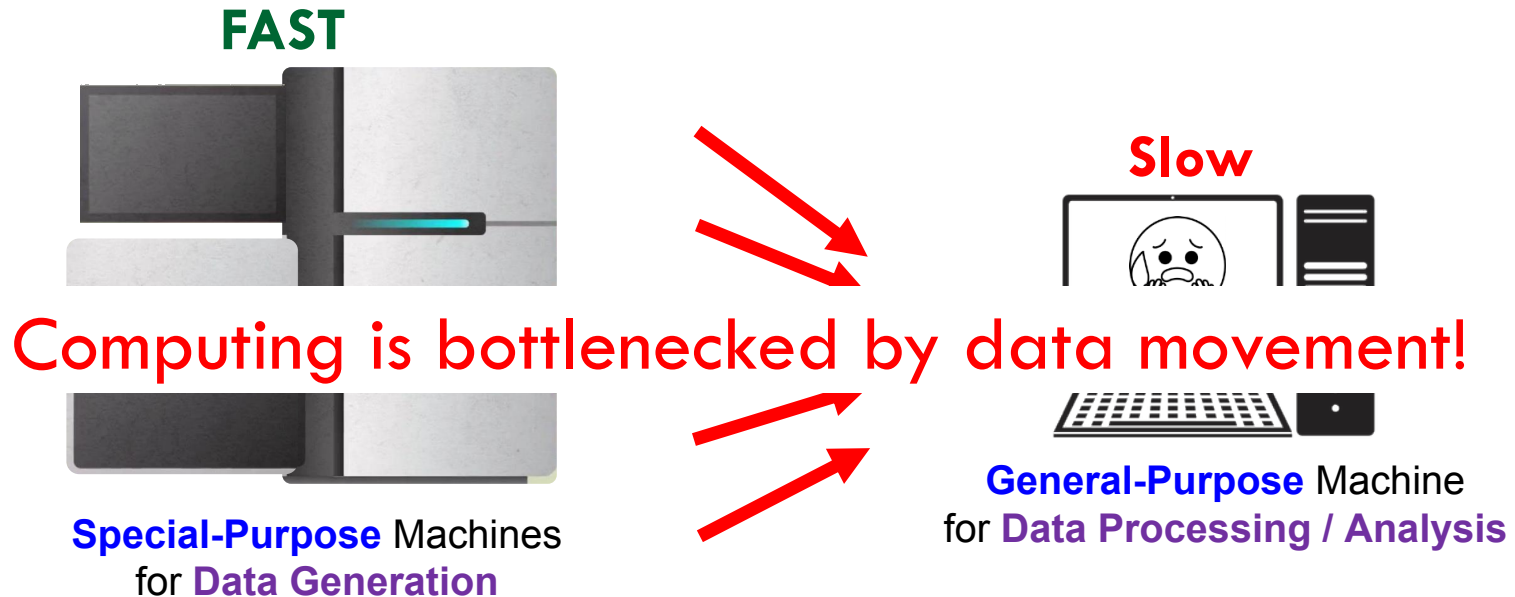
Again! This is not only happened on NN!

- Genome analysis
 - Starts with sequencing random short DNA/RNA fragments of copies of the original molecule.
 - Unfortunately, these reads lack information about their order and which part of genome they are originated from.
 - The second step is to map these reads to a long reference genome.



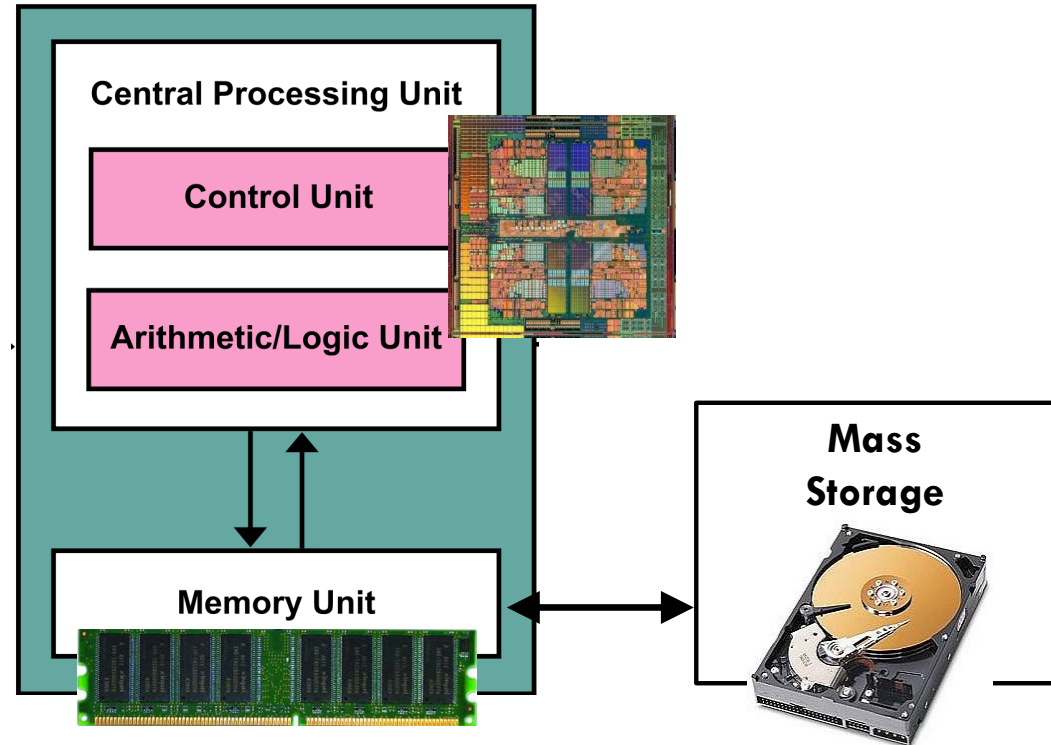
Problems with Data Processing/Analysis Today

- Many “data generators” & “analyzers” are similar to the fig below
 - **Slow** and **inefficient** processing capability.
 - Large amounts of **data movement** are needed.



Conventional Computing System

- In 1945, the architecture of today's computing system is proposed by John von Neumann
- Three key components
 - A **processing unit** – for handling computation
 - A **control unit** – for dealing with communication
 - **Memory** – for storing data and instructions to be used
 - Mass storage – for maintaining
- Such **process-centric design** brings hidden dangers



The Dangers of Processor-Centric Design

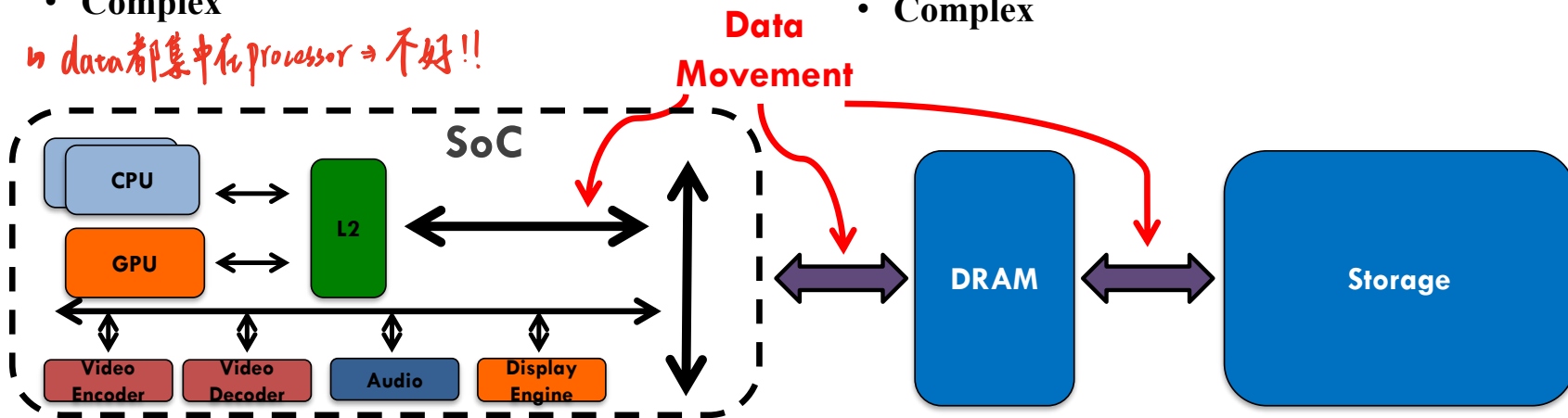
- Grossly-imbalanced systems

- Processing done only in **one place**
- Everything else just stores and moves data: **data moves a lot**
 - Energy inefficient
 - Low performance
 - Complex

↳ data都集中在processor ⇒ 不好!!

- Overly complex and bloated processor (and accelerators)

- To tolerate data access from memory
- Complex hierarchies and mechanisms
 - Energy inefficient
 - Low performance
 - Complex

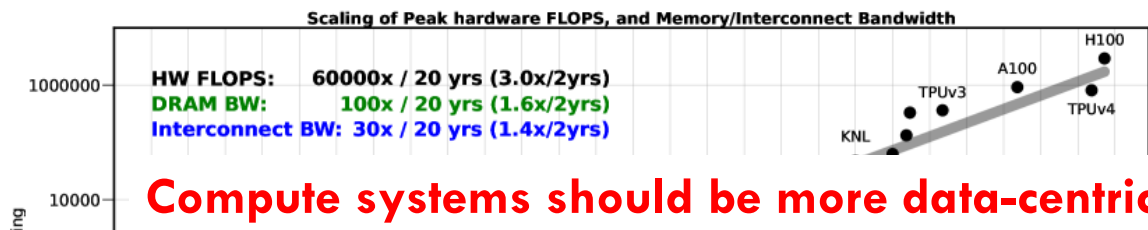


Memory Wall

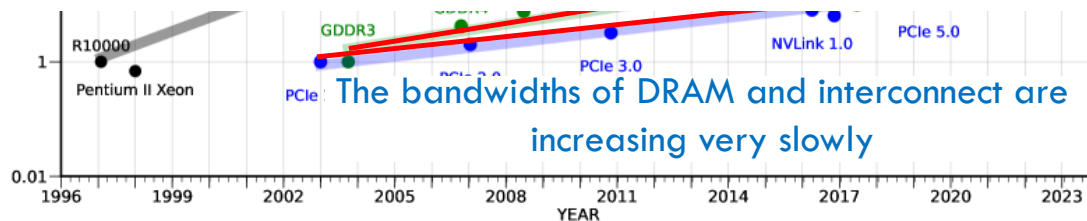
Source: "AI and Memory Wall," <https://arxiv.org/html/2403.14123v1>

- **Memory wall problem**

- *The time to complete an operation is dependent on how fast we can perform the arithmetic as well as how fast we can feed data to the arithmetic units of hardware.*
- No matter how fast the hardware could perform arithmetic ops per second, the problem will be entirely limited by DRAM bandwidth.



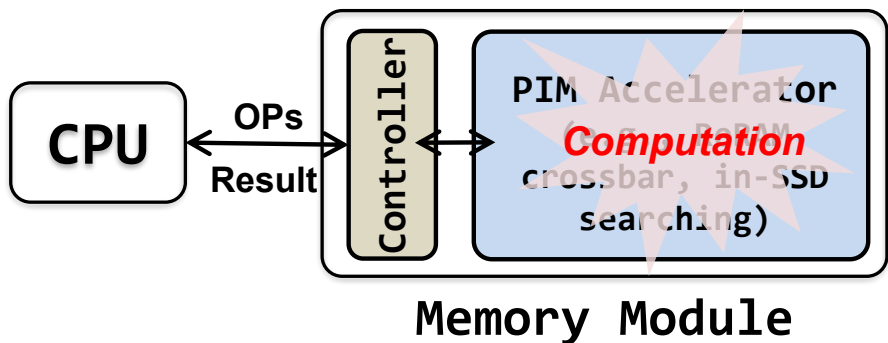
**Processing-In-Memory proposes computing where it makes sense
(where data resides)**



Processing-in-memory (PIM)

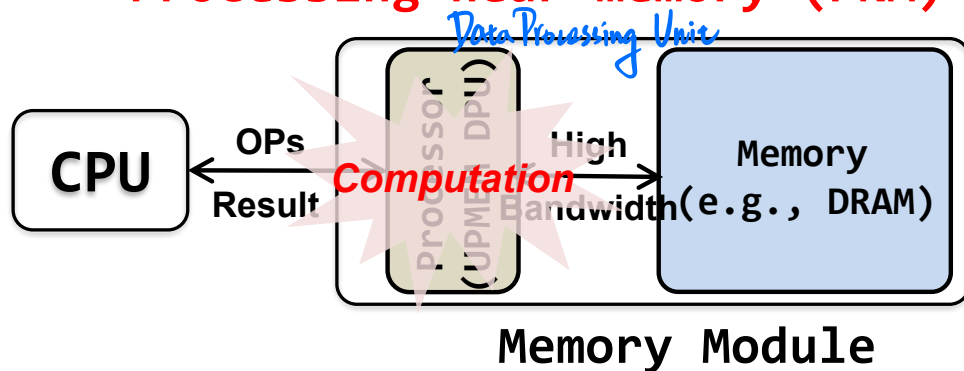
- Modern PIM architectures
 - Processing using memory (PUM), e.g., neural network computation on ReRAM.
 - Processing near memory (PNM): e.g., **UPMEM DPU**, SAMSUNG AXDIMM.
 - Both of them perform computation tasks inside the memory module

Processing using memory (PUM)



↳ read當下先做部分計算 (在mem)

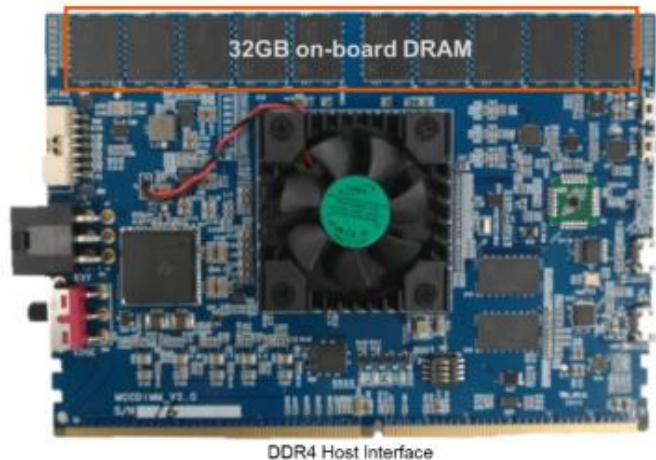
Processing near memory (PNM)



↳ 就近在DPU上先算

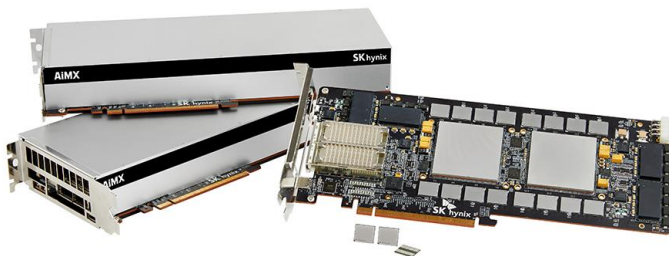
Existing PIM Devices

FPGA+HBM (recommendation
systems and database operators)



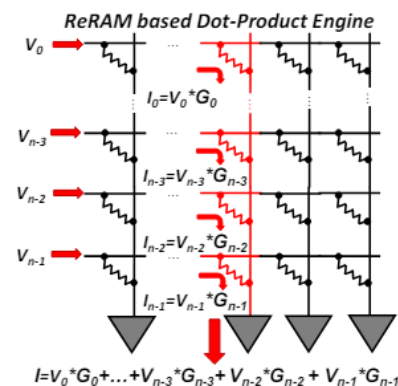
Samsung AXDIMM (PNM) [2]

Deep learning applications



SK Hynix AiM (PNM) [3]

MAC operations



ReRAM Crossbar
(PUM or CIM) [4]

[2] Lee, Donghun, et al. "Improving in-memory database operations with acceleration DIMM (AxDIMM)." Proceedings of the 18th International Workshop on Data Management on New Hardware. 2022.

[3] <https://news.skhynix.com/sk-hynix-debuts-first-gddr6-aim-accelerator-card-aimx-for-generative-ai/>

[4] Wen, Wen, Youtao Zhang, and Jun Yang. "ReNEW: Enhancing lifetime for ReRAM crossbar based neural network accelerators." 2019 IEEE 37th International Conference on Computer Design (ICCD). IEEE, 2019.

Example of PUM: ReRAM Crossbar Array

PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory

Publisher: **IEEE**

[Cite This](#)

[PDF](#)

Ping Chi ; Shuangchen Li ; Cong Xu ; Tao Zhang ; Jishen Zhao ; Yongpan Liu ; Yu Wang ; Yuan Xie [All Authors](#)

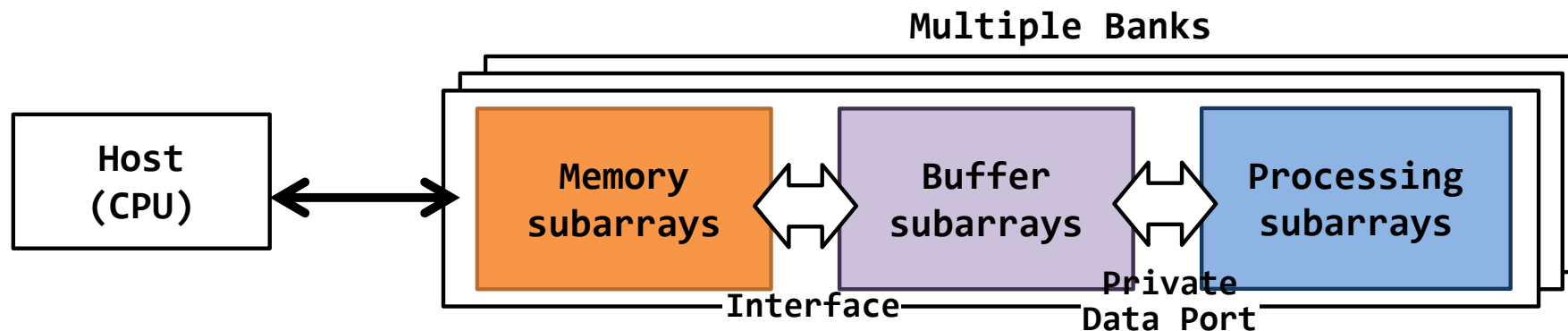
ISAAC: a convolutional neural network accelerator with in-situ analog arithmetic in crossbars

Authors:  [Ali Shafiee](#),  [Anirban Nag](#),  [Naveen Muralimanohar](#),  [Rajeev Balasubramonian](#),  [John Paul Strachan](#),

 [Miao Hu](#),  [R. Stanley Williams](#),  [Vivek Srikumar](#) [Authors Info & Claims](#)

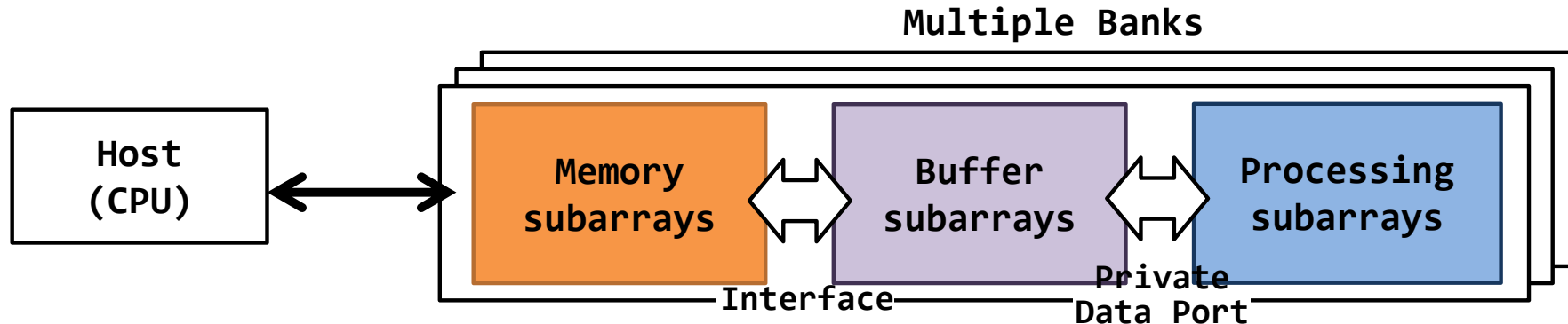
PIM Architecture (1/3)

- Some emerging technology based NVM can perform logic operation beyond storage
 - PCM, STT-MRAM, **ReRAM**
 - It allows NVMs to serve for both **computation and memory**, enabling the PIM architecture



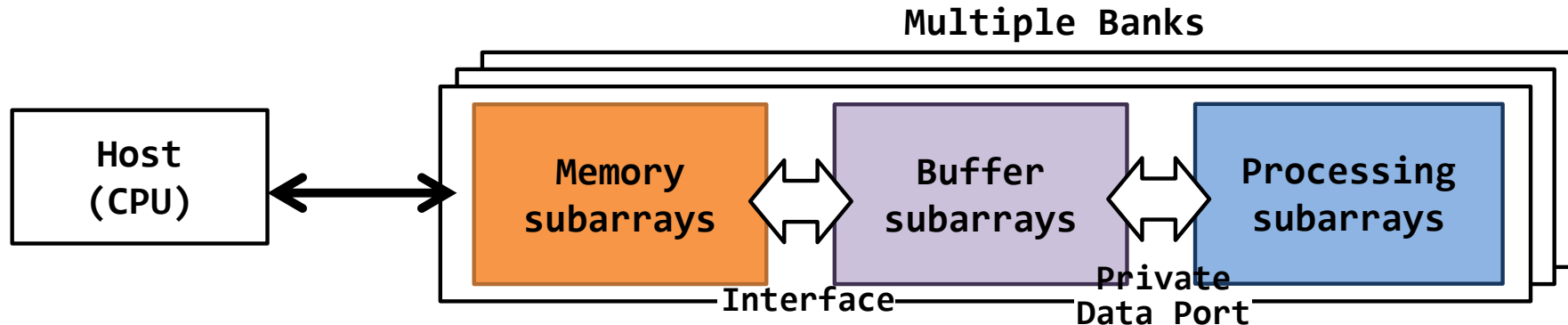
PIM Architecture (2/3)

- A commonly used PIM architecture consists of several components
 - **Memory arrays** serve as conventional memory
 - **Buffer arrays** serve as data buffer
 - **Processing arrays** are utilized to process the data
 - Private data ports are introduced between buffer subarrays and processing subarrays for high data bandwidth
 - Isolated from main memory bus, and hence does not consume bandwidth of main memory

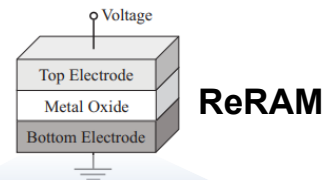


PIM Architecture (3/3)

- PIM best exploits the data locality by **reconfiguring part of the memory blocks** as processing units
 - Data can be directly accessed and transmitted internally without any external memory interfaces
 - Provide large memory bandwidth and fast data access
 - Reduce the data moving energy



What is ReRAM?



- ReRAM is a kind of non-volatile memory

- It is a resistive-based memory

- Write data by pulsing current dQ/dt
- Read data by detecting resistance R

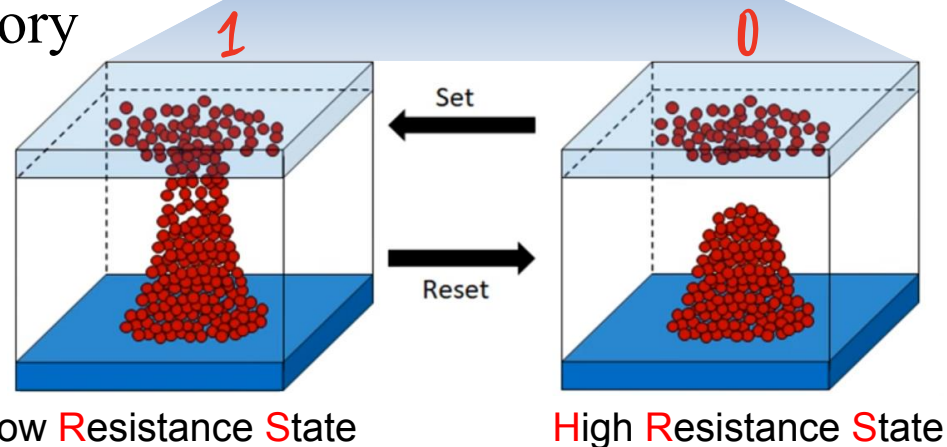
- Very-high density

- Long endurance

- Read latency is comparable to DRAM

- After optimization, write latency is also comparable to DRAM

- *In addition to the data storage, ReRAM can also be used for logic operation and computation, neuromorphic computing, and various sensing application.*



Using ReRAM Crossbar Array for NN Acceleration

- There are lots of convolution in NNs
 - Convolution is using a 'kernel' to extract certain 'features' from an input image.

Input Image (6x6)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

kernel (3x3)

1	-1	-1
-1	1	-1
-1	-1	1

↳ train出來的

=

Output (4x4)

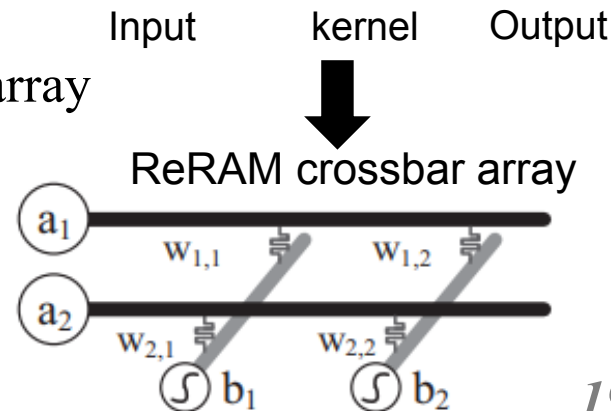
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Data needs to be moved from DRAM to CPU for finishing the calculation

- ReRAM crossbar array

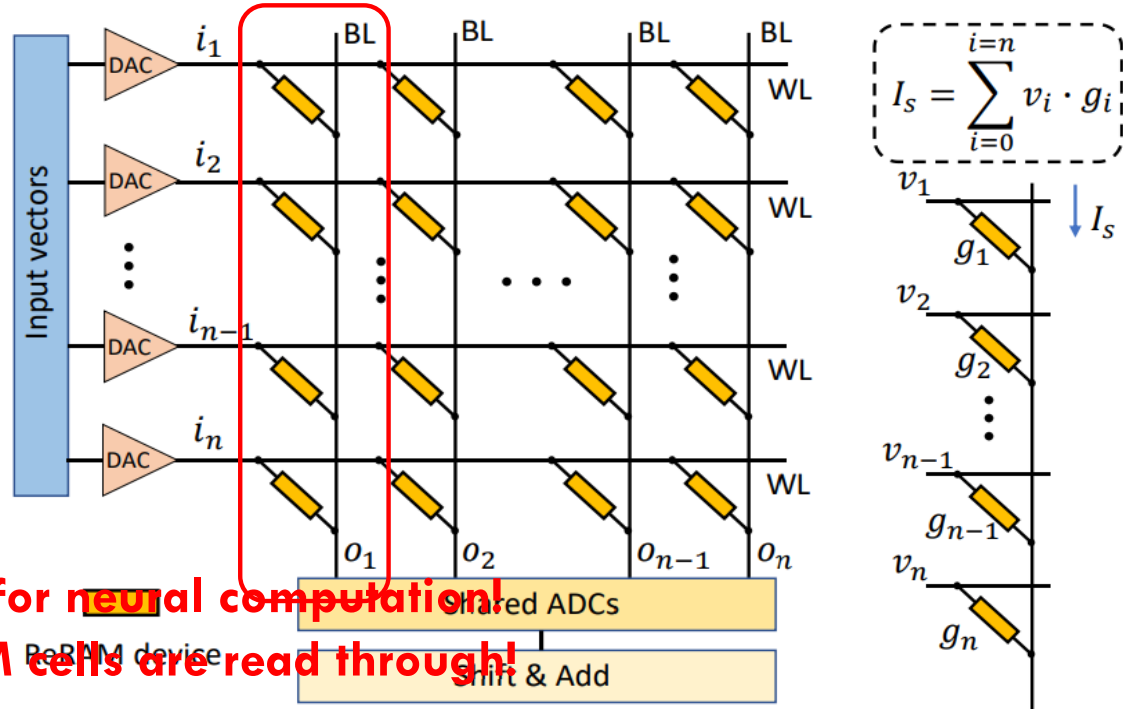
- Map and store parameters of kernel to ReRAM crossbar array
- Translate inputs into different input voltages
- Sum up the current and turn it into output values

$$[a_1 \quad a_2] \times \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$



ReRAM Crossbar Array for Matrix-Vector Multiplication

- The input vectors are fed into the crossbar array as wordline (WL) supply voltage
- Based on Kirchhoff's law, the current summed at each bitline (BL) results the matrix-vector multiplication.
- Multiple cells connecting to the same WL are used to represent one parameter value
- Shift & add blocks are used to sum up the computation result from different columns



Using a ReRAM crossbar array for neural computation!
Calculation finishes once ReRAM cells are read through!

Example of PUM:

In-Memory-Searching on 3D NAND

In-Memory-Searching Architecture Based on 3D-NAND Technology with Ultra-high Parallelism

Publisher: IEEE

[Cite This](#)

[PDF](#)

Po-Hao Tseng ; Feng-Ming Lee ; Yu-Hsuan Lin ; Liang-Yu Chen ; Yung-Chun Li ; Han-Wen Hu ; Yun-Yuan Wang ; Chih-Chang Hsieh ; Ming-Hsi... [All Authors](#)

A 512Gb In-Memory-Computing 3D-NAND Flash Supporting Similar-Vector-Matching Operations on Edge-AI Devices

Publisher: IEEE

[Cite This](#)

[PDF](#)

Han-Wen Hu ; Wei-Chen Wang ; Chung-Kuang Chen ; Yung-Chun Lee ; Bo-Rong Lin ; Huai-Mu Wang ; Yen-Po Lin ; Yu-Chao Lin ; Chih-Chang ... [All Authors](#)

SLC and MLC In-Memory-Approximate-Search Solutions in Commercial 48-layer and 96-layer 3D-NAND Flash Memories

Publisher: IEEE

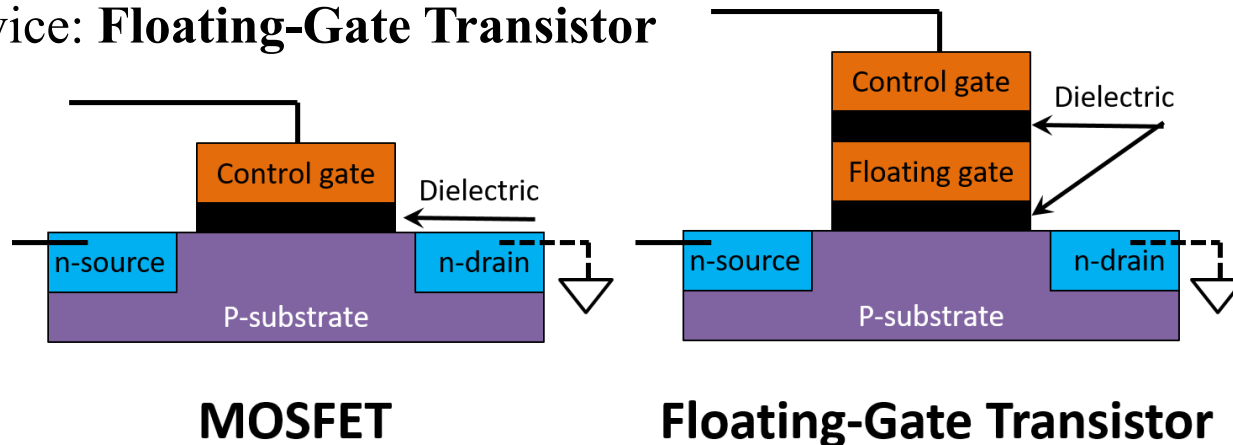
[Cite This](#)

[PDF](#)

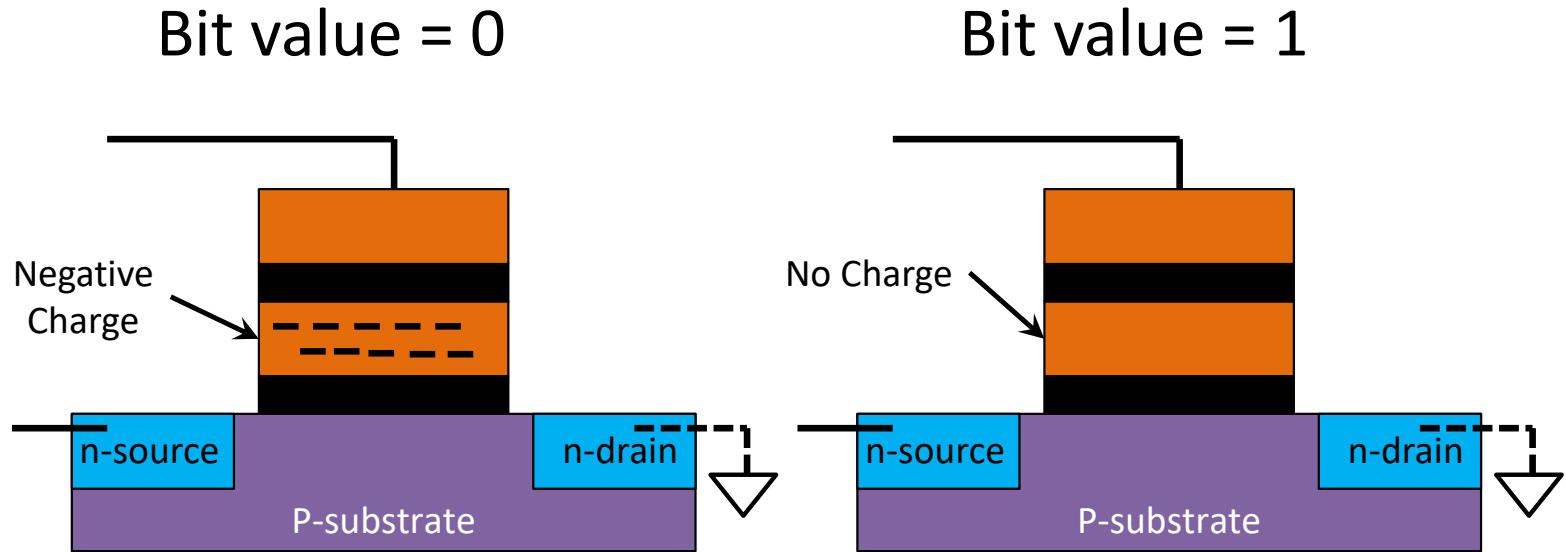
Po-Hao Tseng ; Tian-Cig Bo ; Yu-Hsuan Lin ; Yu-Chao Lin ; Jhe-Yi Liao ; Feng-Ming Lee ; Yu-Yu Lin ; Ming-Hsiu Lee ; Kuang-Yeu Hsieh ; Keh-C... [All Authors](#)

Flash Memory

- Flash memory is a widely used memory/storage technology in today's products
 - E.g., USB drives, solid-state drives and mobile devices
- Flash memory is a type of **non-volatile memory**
 - It holds data states even when the power is lost.
- Nice features:
 - Shock-resistance, non-volatility, low energy consumption, small feature size, low cost
- The Key Device: **Floating-Gate Transistor**



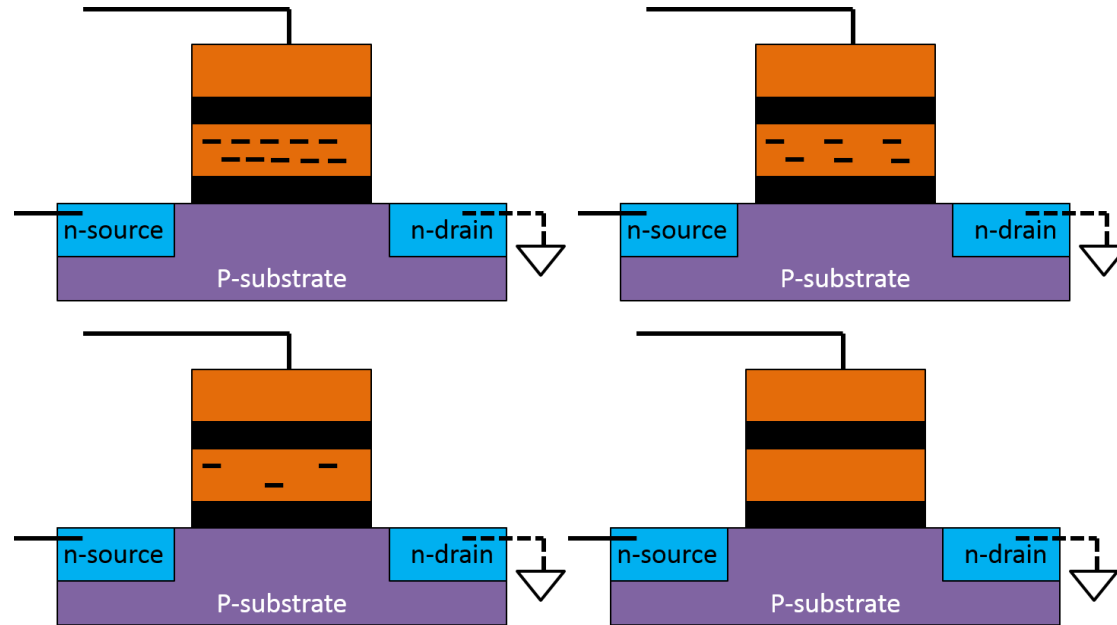
Single Level Cell



SLC (Single Level Cell)

- 2 charge states
- 1 bit per floating gate transistor

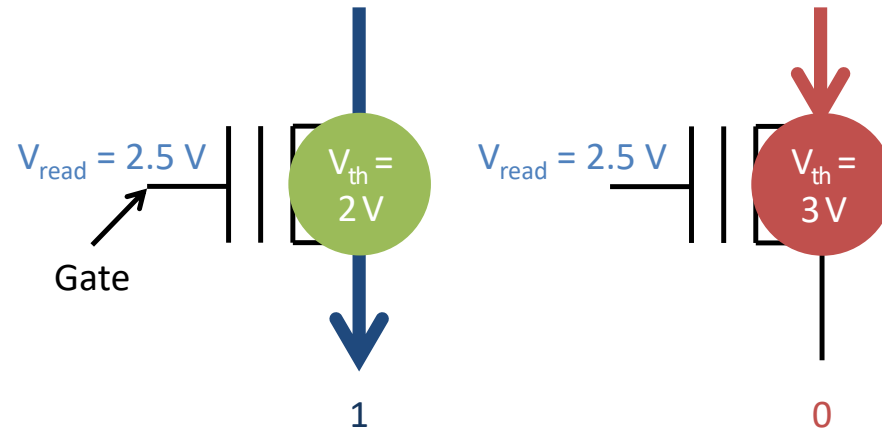
Multi Level Cell



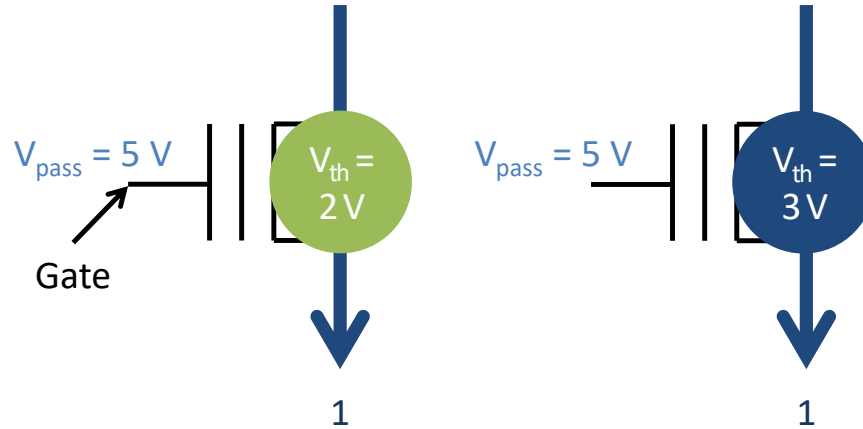
MLC (Multi Level Cell)

- > 2 charge states
- > 1 bit per floating gate transistor

Flash Read

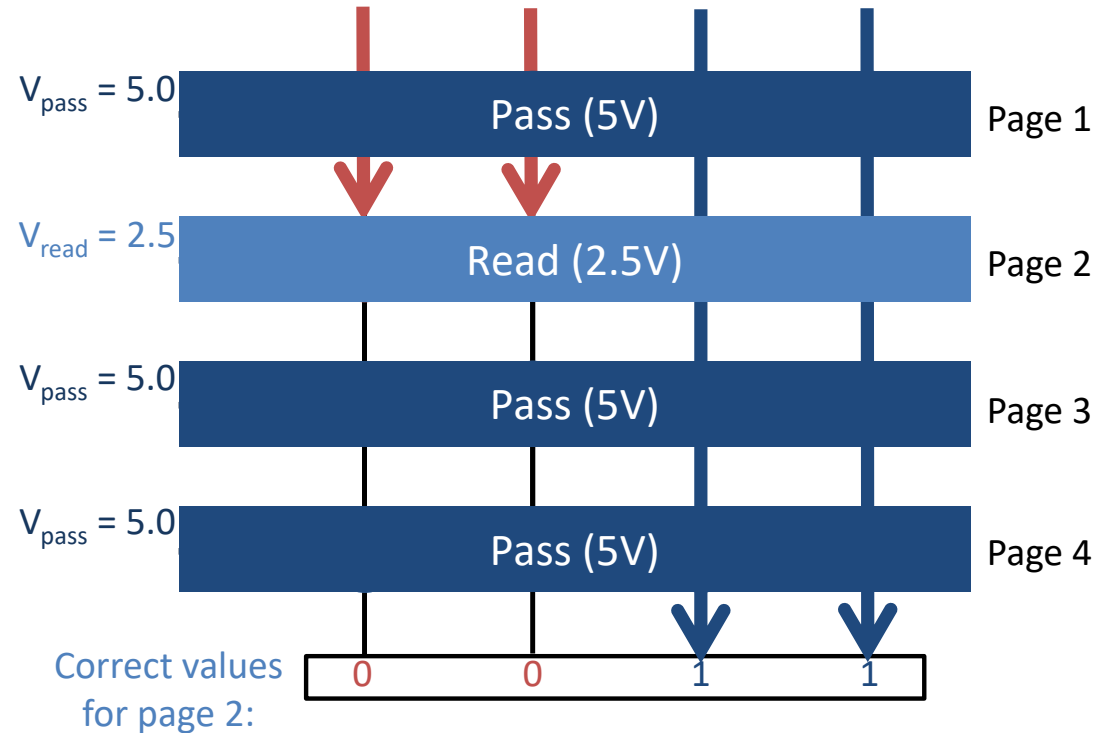


Flash Pass-Through



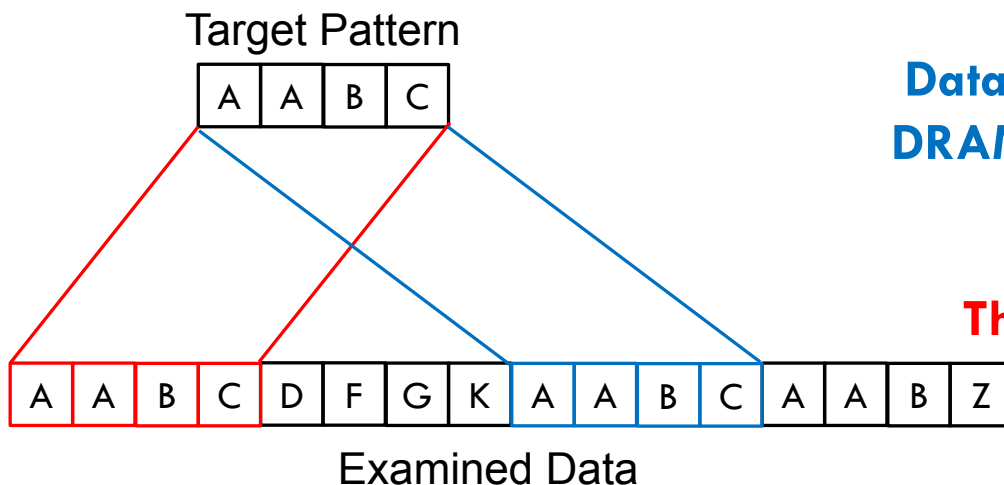
Read from Flash Cell Array

↳ R/W: wordline



Pattern Matching

- Pattern matching is a computational technique used to identify patterns within data.
 - **Searching for a specified pattern within a larger dataset or text**
 - It can be applied across various domains
 - Text analysis, image processing, bioinformatics, information security, speech recognition, and data compression



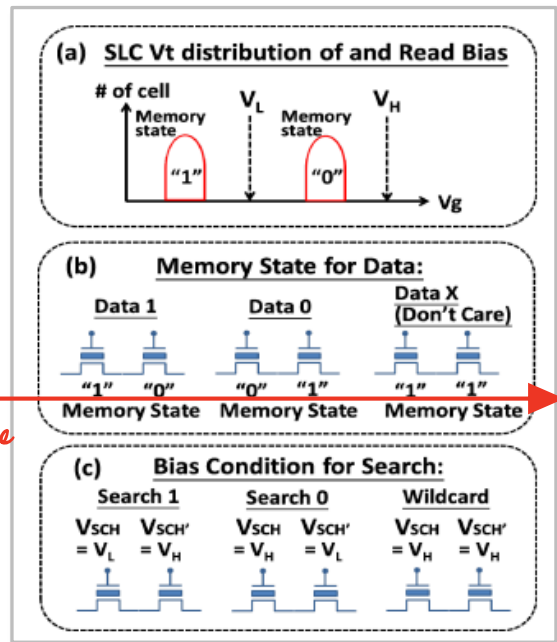
Data needs to be moved from HDD to DRAM, and from DRAM to CPU for finishing the pattern matching.

There are many to-be-examined data!

Pattern Matching via In-Memory-Searching Architecture based on 3D NAND

- In-memory-search architecture allow the specific pattern matching to be done by reading flash cells
 - It provides three memory states for data written
 - Data "1", Data "0", Data X (Don't care) $\Rightarrow$ 存在兩個cell上
 - It also provides specific search operation to test the stored values on the examined flash cells

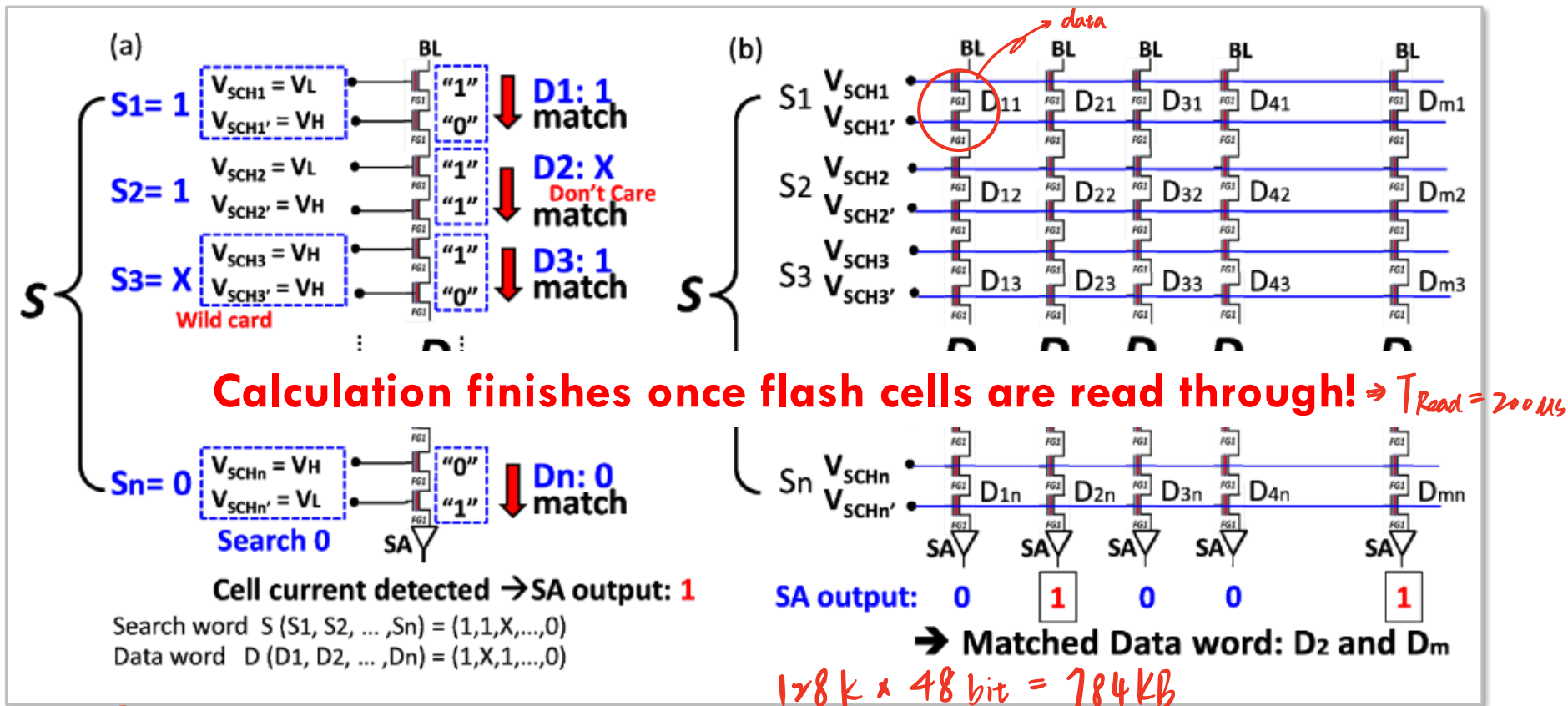
	Search1	Search0	SearchX (Wildcard)
Data1	V_L V_H "1" "0" $\rightarrow$ Current MATCH	V_H V_L "1" "0" $\times$ No current MISMATCH	V_H V_H "1" "0" $\rightarrow$ Current MATCH
Data0	V_L V_H "0" "1" $\times$ No current MISMATCH	V_H V_L "0" "1" $\rightarrow$ Current MATCH	V_H V_H "0" "1" $\rightarrow$ Current MATCH
DataX (Don't Care)	V_L V_H "1" "1" $\rightarrow$ Current MATCH	V_H V_L "1" "1" $\rightarrow$ Current MATCH	V_H V_H "1" "1" $\rightarrow$ Current MATCH



bit line

IMS unit cell

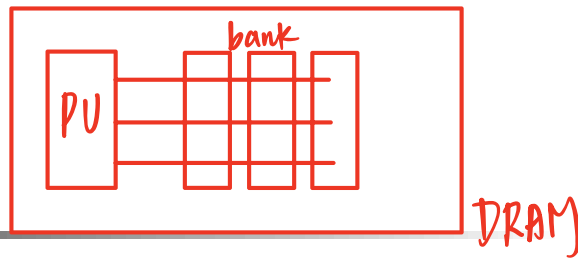
How to Perform the In-Memory Searching



128k x 48 bit = 784KB
(BL) (96MB/2)

速度快 缺 BL 能存的 bit 太少

Example of PNM: UPMEM DPU



Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System

Publisher: **IEEE**

[Cite This](#)

[PDF](#)

Juan Gómez-Luna ; Izzat El Hajj ; Ivan Fernandez ; Christina Giannoula ; Geraldo F. Oliveira ; Onur Mutlu [All Authors](#)

UpPipe: A Novel Pipeline Management on In-Memory Processors for RNA-seq Quantification

Publisher: **IEEE**

[Cite This](#)

[PDF](#)

Liang-Chi Chen ; Chien-Chung Ho ; Yuan-Hao Chang [All Authors](#)

PIMDup: An Optimized Deduplication Design on a Real Processing-in-Memory System

Publisher: **IEEE**

[Cite This](#)

[PDF](#)

Chun-Le Yeh ; Liang-Chi Chen ; Chien-Chung Ho ; Yu-Ming Chang ; Da-Wei Chang [All Authors](#)

UPMEM Processing-in-DRAM Engine (2019)

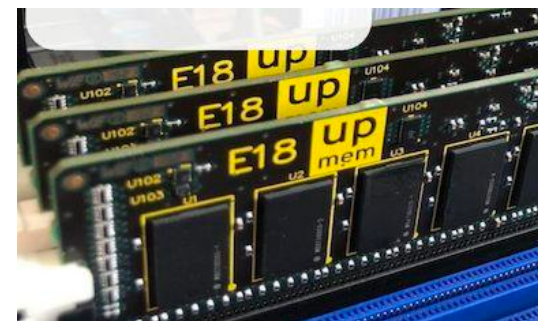
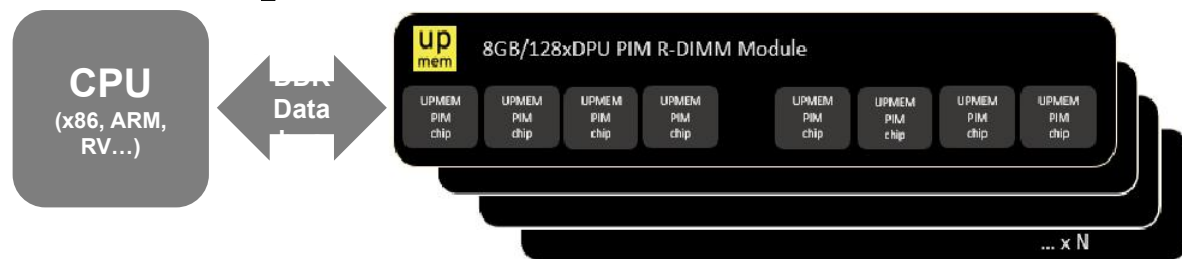
- Processing in DRAM Engine

<https://www.anandtech.com/show/14750/hot-chips-31-analysis-inmemory-processing-by-upmem>
<https://www.upmem.com/video-upmem-presenting-its-true-processing-in-memory-solution-hot-chips-2019/>

- Includes **standard DIMM modules**, with a **large number of DPU processors** combined with DRAM chips.

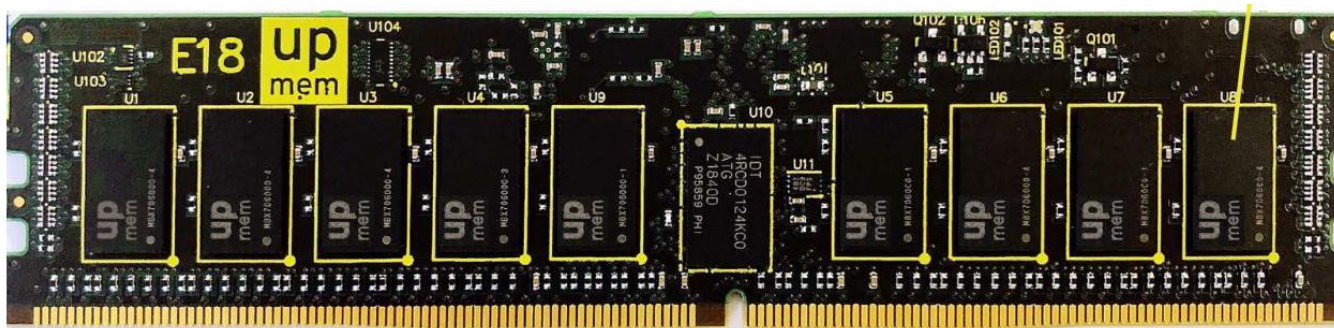
- UPMEM DIMMs

- **Replaces standard DIMMs**
- **DDR4 R-DIMM modules**
 - 8GB+128 DPUs (16 PIM chips)
 - Standard 2x-nm DRAM process
- **Large amounts of compute & memory bandwidth**
 - 2TB/s DRAM-DPU BW for 128GB+2048 DPUs config
- **Easily programmable SDK: C-programmable**



UPMEM DIMMs

- Standard DRAM package & DIMM (DDR4 2400 R-DIMM module)



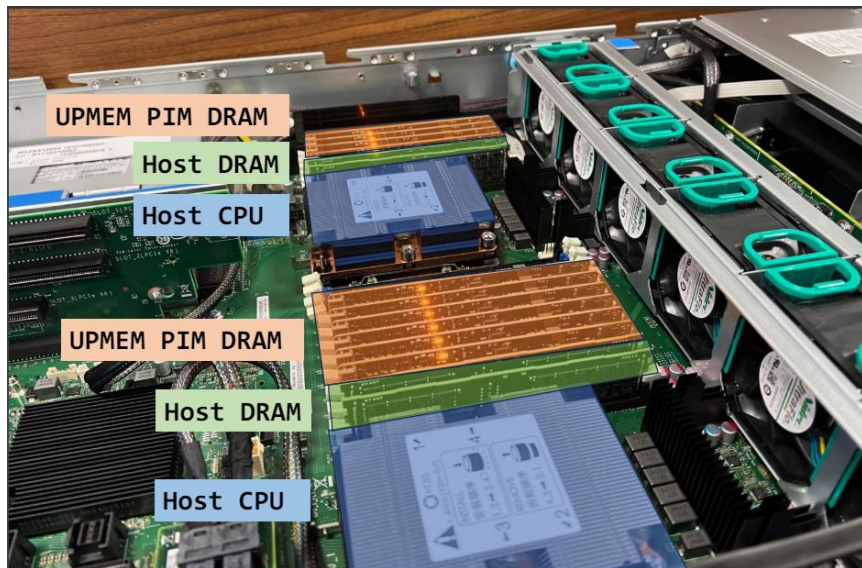
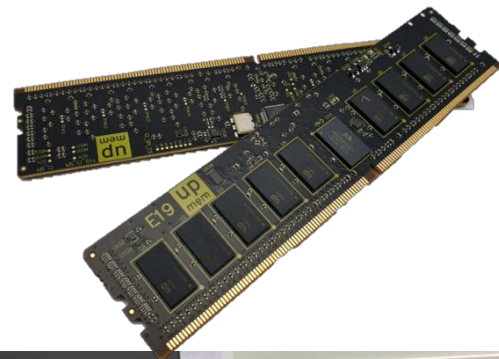
4Gb DRAM DDR4 2400+ 8DPUs
@ 500MHz
1GB/s DRAM-DPU bandwidth
Standard DRAM package~1cm²
die – ~1,2W

– Different types of modules

- E19: 8 chips/DIMM (1 rank). DPUs @ 267 MHz
- P21: 16 chips/DIMM (2 ranks). DPUs @ 350 MHz

UPMEM DRAM Processing Units (DPU)

- UPMEM DPU is the first commercial general PIM system
- The UPMEM is compatible with modern computer's DDR DIMM slots



A server with ubuntu and the UPMEM DPU's driver + SDK

UPMEM PIM massive benefits

- Massive speed-up
 - Massive additional compute & bandwidth
- Massive energy gains
 - Most data movement on chip
- Low cost
 - ~300\$ of additional DRAM silicon
 - Affordable programming
- Massive ROI / TCO gains

Energy efficiency when computing on or off memory chip		Server + PIM DRAM	Server + normal DRAM
DRAM to processor 64-bit operand	pJ	~150	~3000*
Operation	pJ	~20	~10*
Server consumption	W	~700W	~300W
speed-up		~ x20	x1
energy gain		~ x10	x1
TCO gain		~ x10	x1

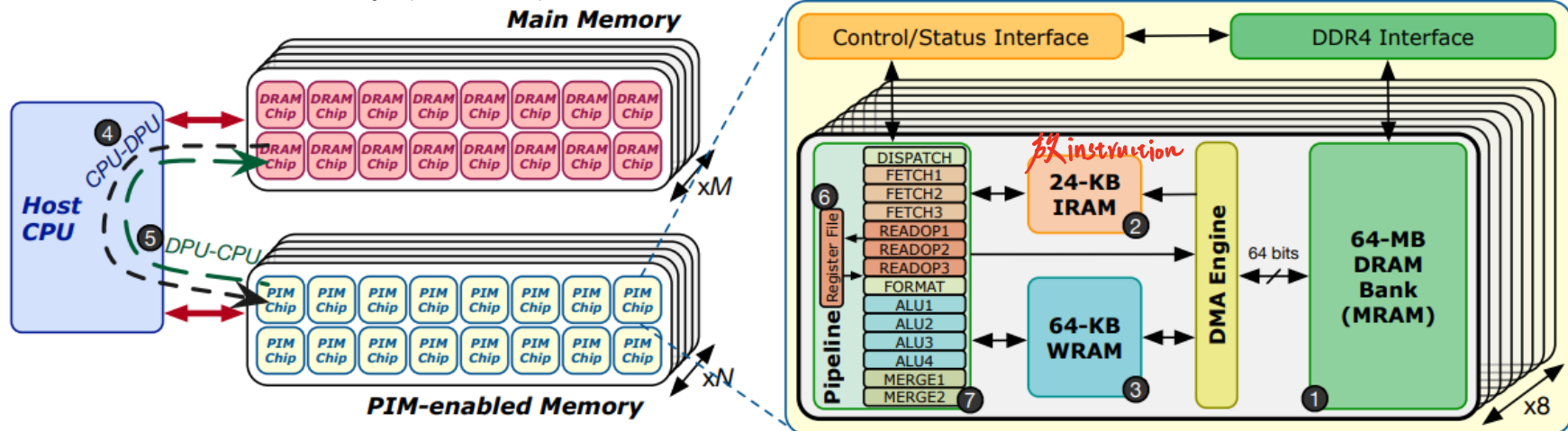
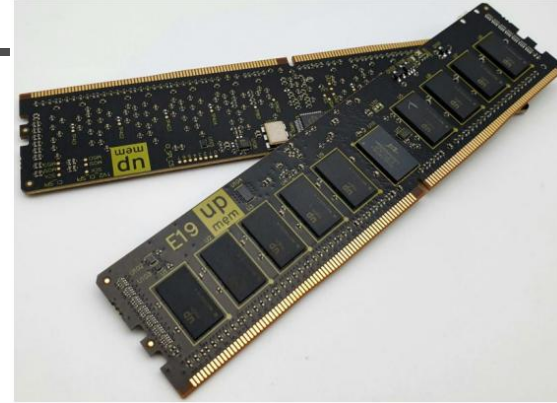
**Exascale Computing Trends: Adjusting to the “New Normal” for Computer Architecture; John Shalf, Computing in Science & engineering, 2013*

The Hurdles on the road to the Graal

- DRAM process highly constrained
 - 3x slower transistors than same node digital process
 - Logic 10 times less dense vs. ASIC process
 - Routing density dramatically lower
 - 3 metals only for routing (vs. 10+), pitch x4 larger
- Strong design choices mandatory
- But the PIM Graal is worth it !

UPMEM DPU

- UPMEM DRAM Processing Units (DPU) [4]
 - Each DPU is a 32-bit RISC core
 - 24 hardware threads (tasklets)
 - 24KB instruction memory (IRAM)
 - 64KB working memory (WRAM)
 - 64MB main memory (MRAM)

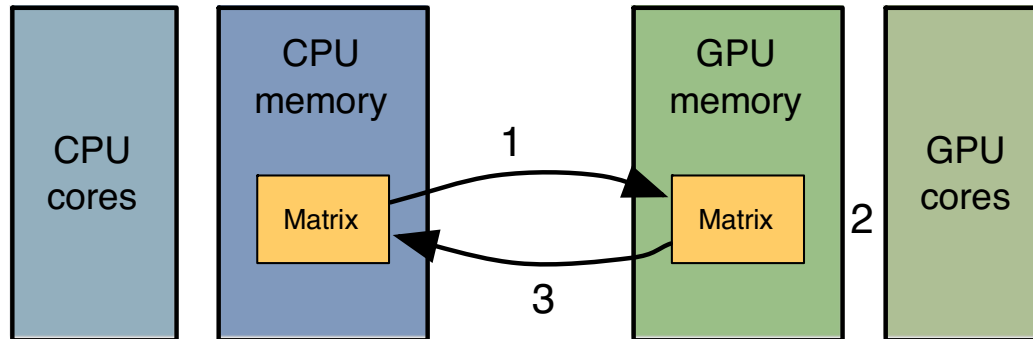


Accelerator Model (I)

- UPMEM DIMMs coexist with conventional DIMMs
- Integration of UPMEM DIMMs in a system follows an **accelerator model**
- UPMEM DIMMs can be seen as a **loosely coupled accelerator**
 - Explicit data movement between the main processor (host CPU) and the accelerator (UPMEM)
 - Explicit kernel launch onto the UPMEM processors
- This resembles GPU computing

GPU Computing

- Computation is **offloaded to the GPU**
- Three steps
 - CPU-GPU data transfer (1)
 - GPU kernel execution (2)
 - GPU-CPU data transfer (3)



Accelerator Model (II)

- FIG. 6 is a flow diagram representing operations in a method of delegating a processing task to a DRAM processor according to an example embodiment

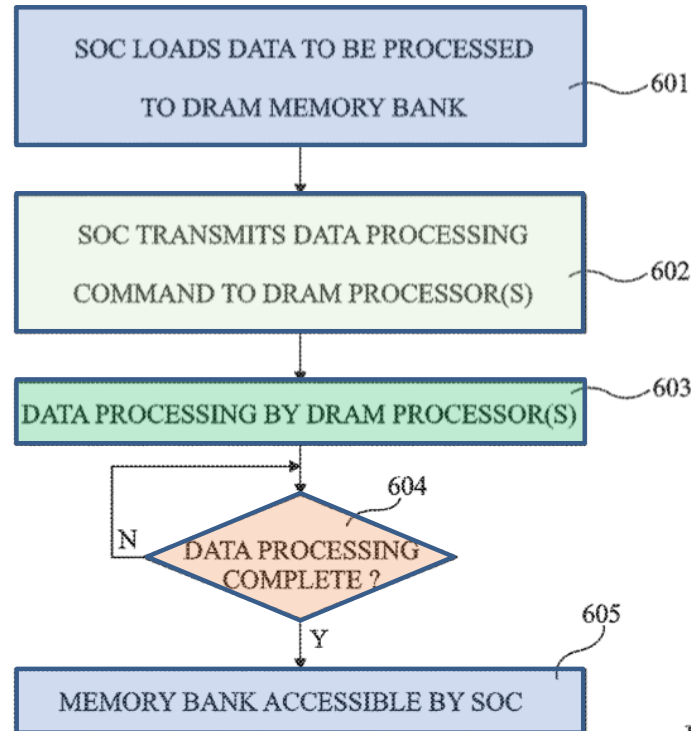
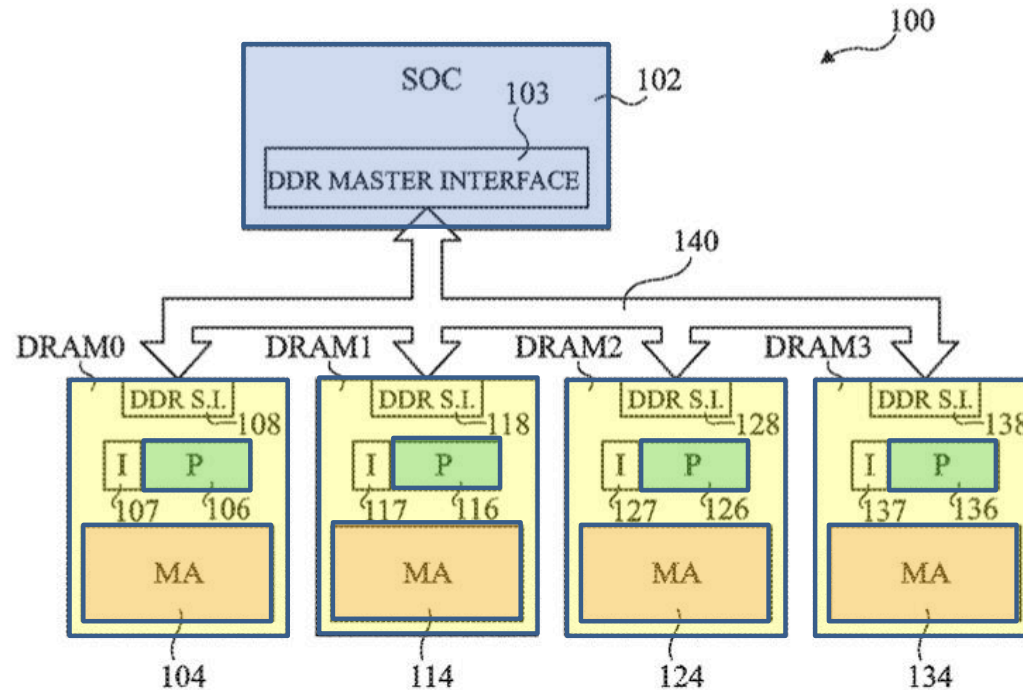


Fig 6

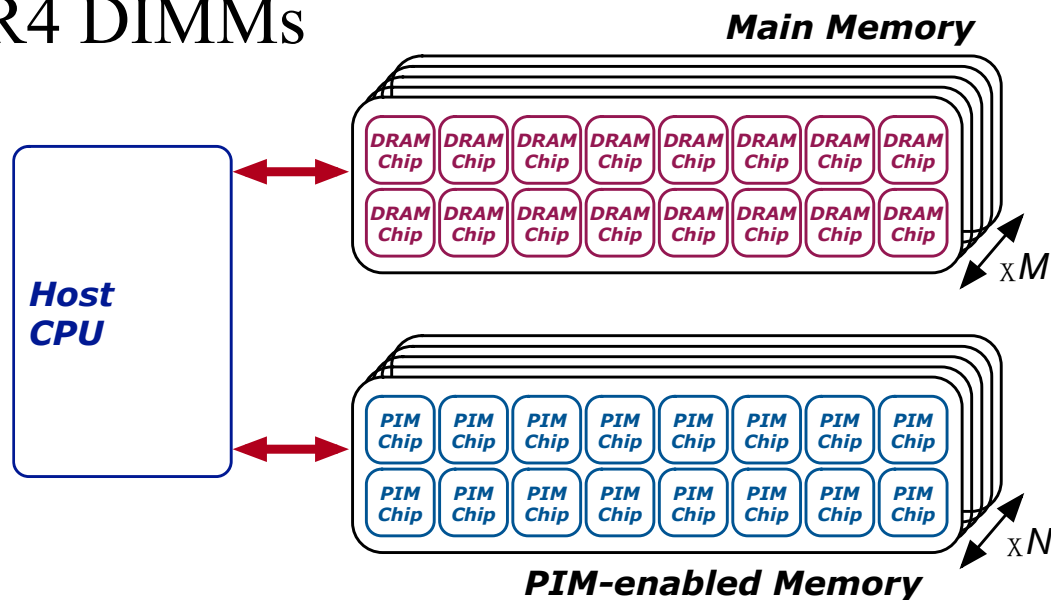
System Organization (I)

- FIG. 1 schematically illustrates a computing system comprising DRAM circuits having integrated processors according to an example embodiment



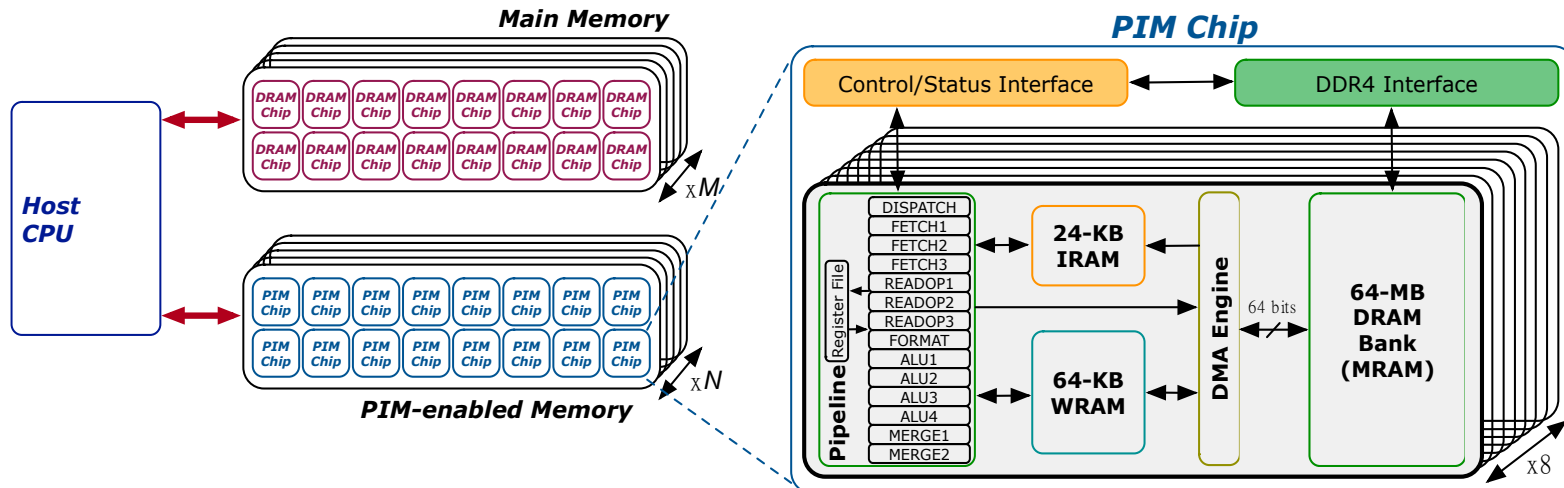
System Organization (II)

- In a UPMEM-based PIM system UPMEM DIMMs coexist with regular DDR4 DIMMs



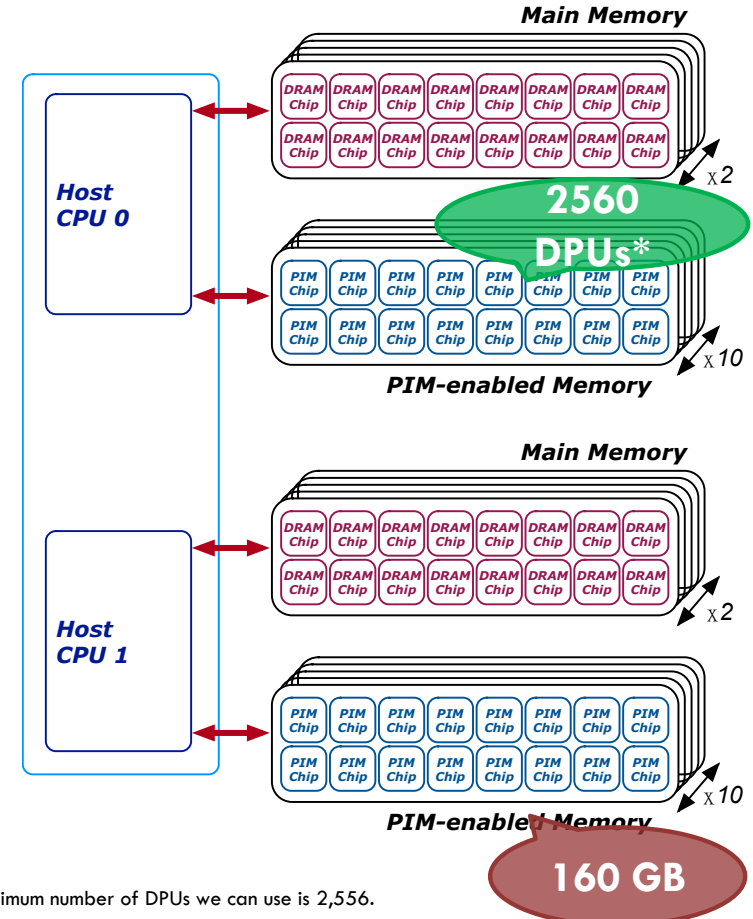
System Organization (III)

- A UPMEM DIMM contains 8 or 16 chips
 - Thus, 1 or 2 ranks of 8 chips each
- Inside each PIM chip there are:
 - 8 64MB banks per chip: Main RAM (MRAM) banks
 - 8 DRAM Processing Units (DPUs) in each chip, 64 DPUs per rank



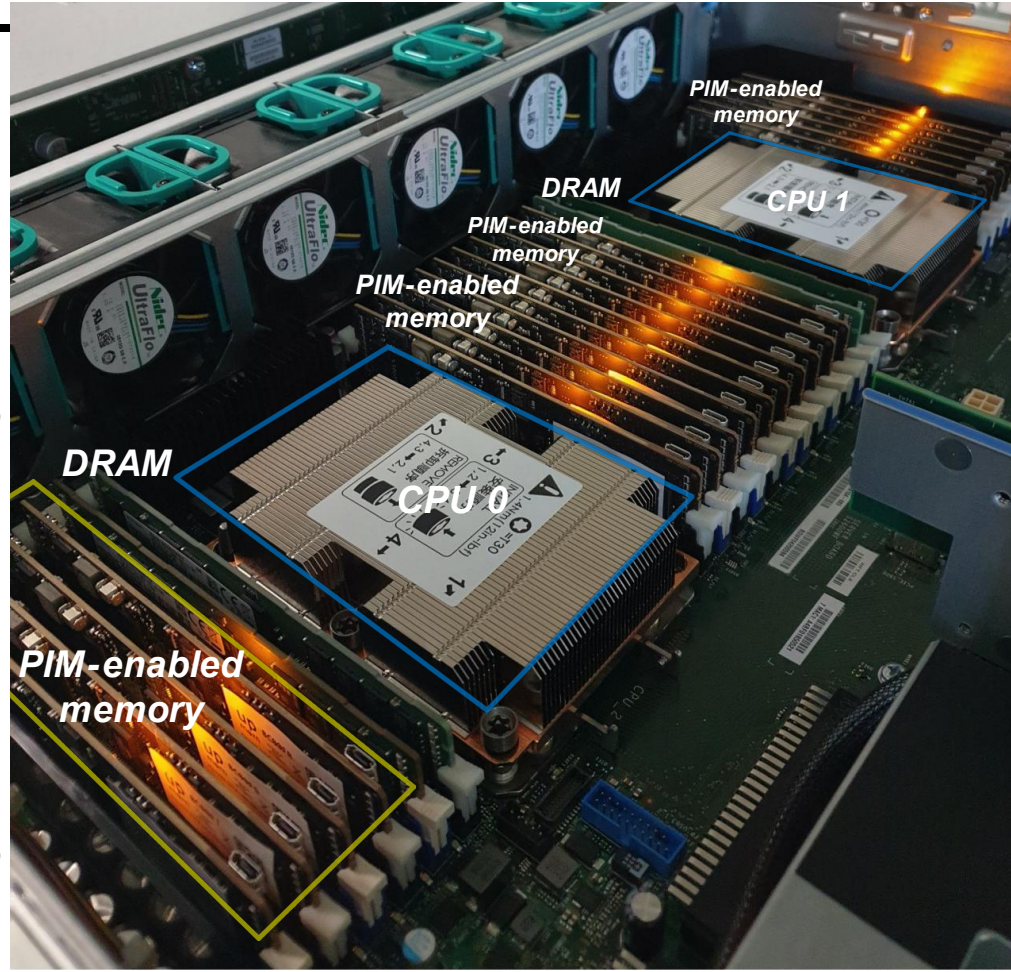
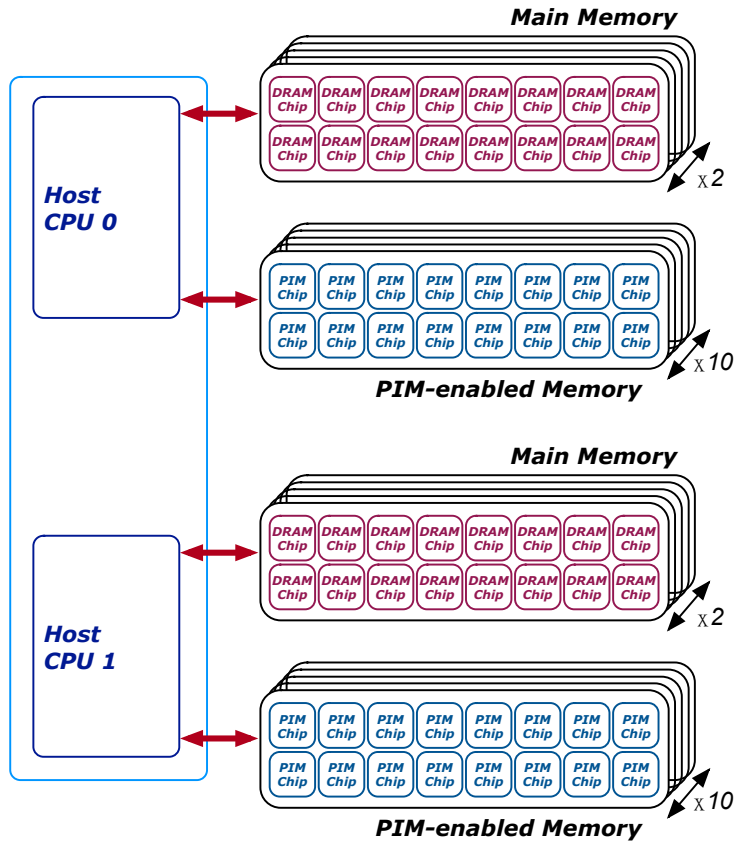
2,560-DPU System (I)

- UPMEM-based PIM system with 20 UPMEM DIMMs of 16 chips each (40 ranks)
 - P21 DIMMs
 - Dual x86 socket
 - UPMEM DIMMs coexist with regular DDR4 DIMMs
 - 2 memory controllers/socket (3 channels each)
 - 2 conventional DDR4 DIMMs on one channel of one controller



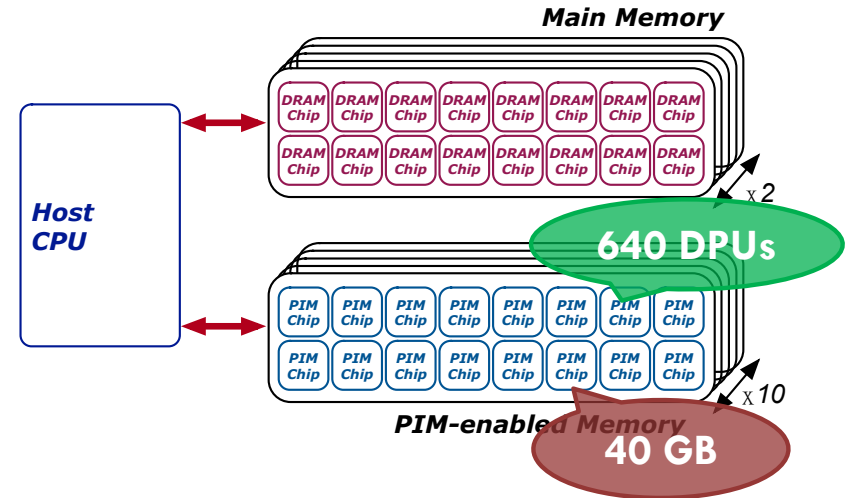
* There are 4 faulty DPUs in the system that we use in our experiments. Thus, the maximum number of DPUs we can use is 2,556.

2,560-DPU System (II)



640-DPU System

- UPMEM-based PIM system with 10 UPMEM DIMMs of 8 chips each (10 ranks)
 - E19 DIMMs
 - x86 socket
 - 2 memory controllers (3 channels each)
 - 2 conventional DDR4 DIMMs on one channel of one controller

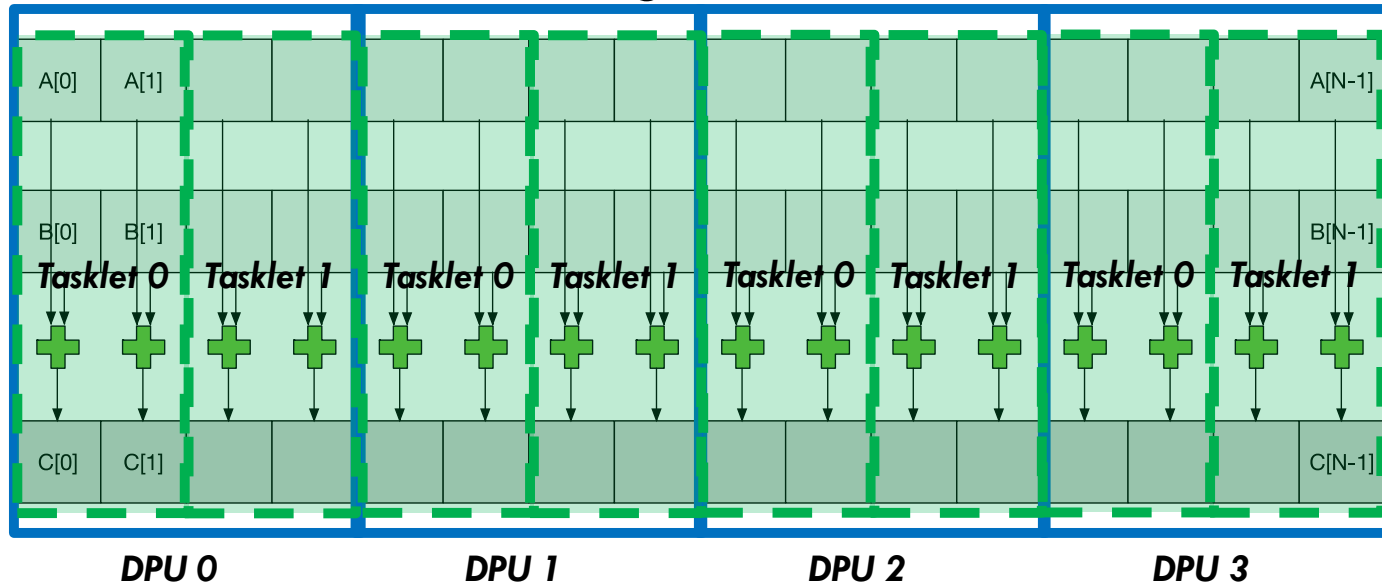


DPU Sharing? Security Implications?

- DPUs cannot be shared across multiple CPU processes
 - There are so many DPUs in the system that there is no need for sharing
- According to UPMEM, this assumption makes things simpler
 - No need for OS
 - Simplified security implications: No side channels

Vector Addition (VA)

- The first programming example
- We partition the input arrays across:
 - DPUs
 - Tasklets, i.e., software threads running on a DPU



General Programming Recommendations

- From UPMEM programming guide*, presentations★, and white papers★

GENERAL PROGRAMMING RECOMMENDATIONS

1. Execute on the *DRAM Processing Units (DPUs)* **portions of parallel code** that are as long as possible.
2. Split the workload into **independent data blocks**, which the DPUs operate on independently.
3. Use **as many working DPUs** in the system as possible.
4. Launch at least **11 tasklets (i.e., software threads)** per DPU.

DPU Allocation

- `dpu_alloc()` allocates a number of DPUs
 - Creates a `dpu_set`

```
1 struct dpu_set_t dpu_set, dpu;  
2 uint32_t nr_of_dpus;  
3  
4 // Allocate DPUs  
5 DPU_ASSERT(dpu_alloc(NR_DPUS, NULL, &dpu_set));  
6  
7 DPU_ASSERT(dpu_get_nr_dpus(dpu_set, &nr_of_dpus));  
8 printf("Allocated %d DPU(s)\n", nr_of_dpus);  
9
```

- Can we allocate different DPU sets over the course of a program?
 - Yes, we can change the number of DPUs in the DPU set as computation progresses
- We deallocate a DPU set with `dpu_free()`

Load DPU Binary

- `dpu_load()` loads a program in all DPUs of a `dpu_set`

```
1 // Define the DPU Binary path as DPU_BINARY here
2 #ifndef DPU_BINARY
3 #define DPU_BINARY "./bin/dpu_code"
4 #endif
5
6 ...
7
8 // Load binary
9 DPU_ASSERT(dpu_load(dpu_set, DPU_BINARY, NULL));
10
```

- Is it possible to launch different kernels onto different DPUs?
 - Yes, it is possible. This enables:
 - Workloads with task-level parallelism
 - Different programs using different DPU sets

CPU-DPU/DPU-CPU Data Transfers

- CPU-DPU and DPU-CPU transfers
 - Between host CPU's main memory and DPUs' MRAM banks

- **Serial CPU-DPU/DPU-CPU** transfers:

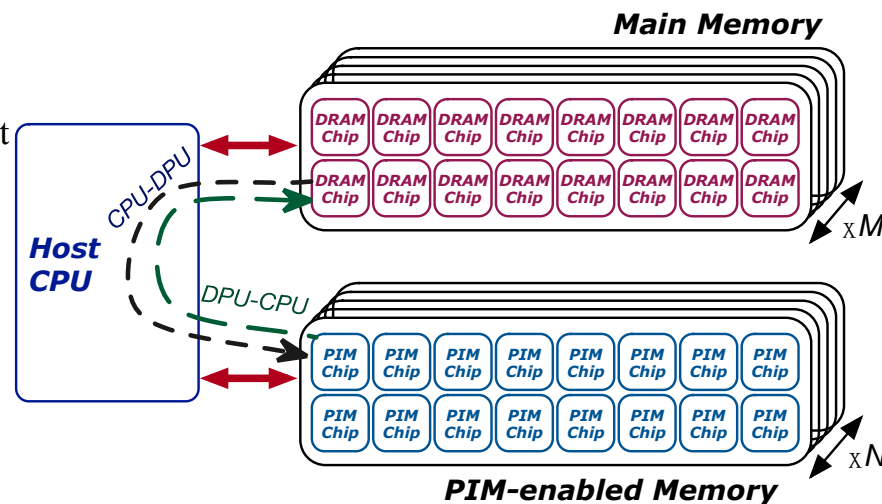
- A single DPU (i.e., 1 MRAM bank)
 - We transfer (part of) a buffer to/from each DPU in the `dpus_set`

- **Parallel CPU-DPU/DPU-CPU** transfers:

- Multiple DPUs (i.e., many MRAM banks)
 - We push different buffers to/from a DPU set in one transfer
 - All buffers need to be of the same size

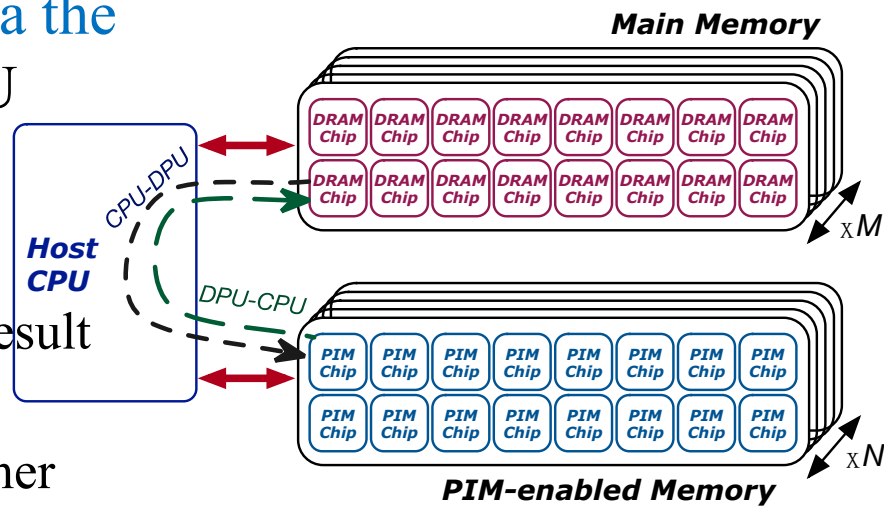
- **Broadcast CPU-DPU** transfers:

- Multiple DPUs with a single buffer
 - We transfer the same buffer to all DPU in the `dpus_set`
- Only CPU to DPU



Inter-DPU Communication

- There is **no direct communication channel between DPUs**
- **Inter-DPU communication takes place via the host CPU** using CPU-DPU and DPU-CPU transfers
- Example communication patterns:
 - Merging of partial results to obtain the final result
 - Only DPU-CPU transfers
 - Redistribution of intermediate results for further computation
 - DPU-CPU transfers and CPU-DPU transfers



How Fast are these Data Transfers?

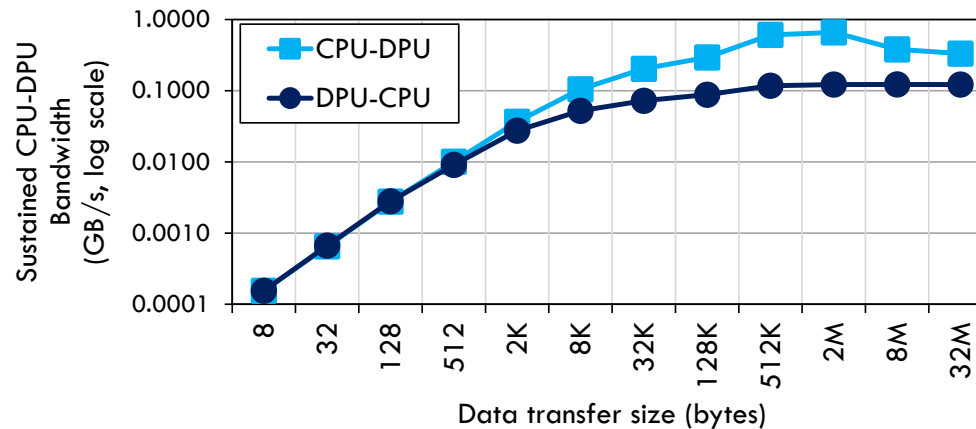
- With a microbenchmark, we obtain the **sustained bandwidth of all types of CPU-DPU and DPU-CPU transfers**
- Two experiments:
 - **1 DPU**: variable CPU-DPU and DPU-CPU transfer size (**8 bytes to 32 MB**)
 - **1 rank**: 32 MB CPU-DPU and DPU-CPU transfers to/from a set of **1 to 64 MRAM banks** within the same rank

DDR4 bandwidth bounds the maximum transfer bandwidth

The cost of the **transfers can be amortized**,
if enough computation is run on the DPUs

CPU-DPU/DPU-CPU Transfers: 1 DPU

- Data transfer size varies between 8 bytes and 32 MB

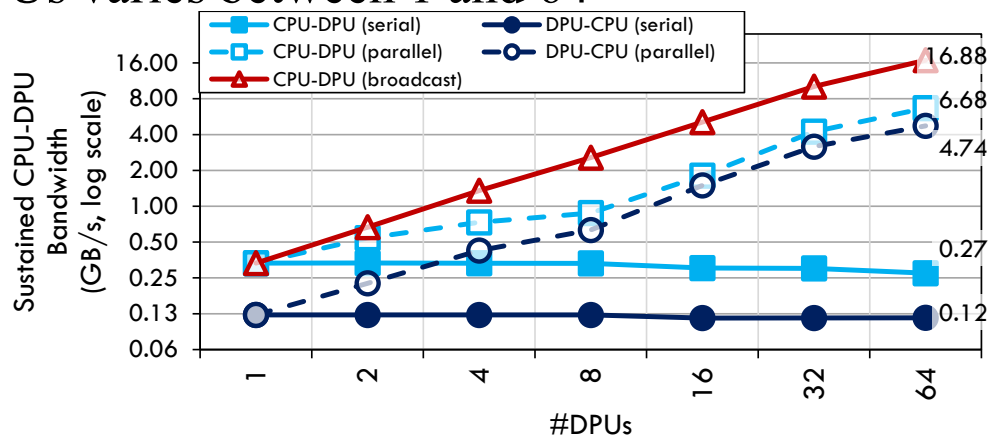


KEY OBSERVATION 7

Larger CPU-DPU and DPU-CPU transfers between the host main memory and the DRAM Processing Unit's Main memory (MRAM) banks **result in higher sustained bandwidth.**

CPU-DPU/DPU-CPU Transfers: 1 Rank (I)

- CPU-DPU (serial/parallel/**broadcast**) and DPU-CPU (serial/parallel)
- The number of DPUs varies between 1 and 64

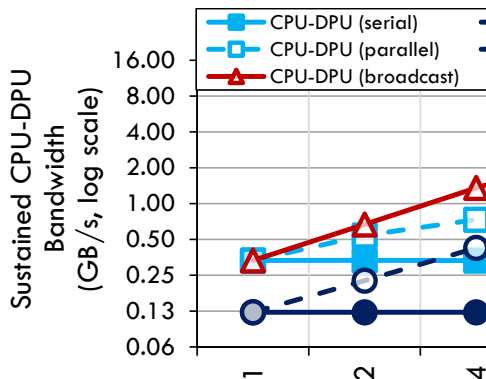


KEY OBSERVATION 8

The **sustained bandwidth of parallel CPU-DPU and DPU-CPU transfers** between the host main memory and the DRAM Processing Unit's Main memory (MRAM) banks **increases with the number of DRAM Processing Units inside a rank.**

CPU-DPU/DPU-CPU Transfers: 1 Rank (II)

- CPU-DPU (serial/parallel/**broadcast**) and DPU-CPU (serial/parallel)
- The number of DPUs varies between 1 and 64



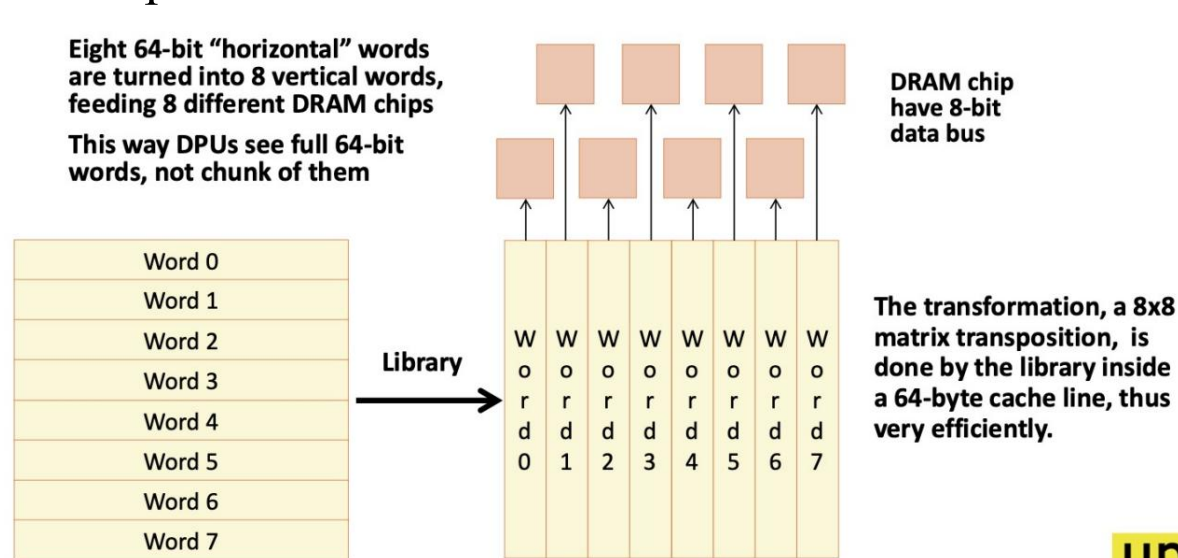
KEY OBSERVATION 9

The sustained bandwidth of parallel CPU-DPU transfers is higher than the sustained bandwidth of parallel DPU-CPU transfers due to different implementations of CPU-DPU and DPU-CPU transfers in the UPMEM runtime library.

The sustained bandwidth of broadcast CPU-DPU transfers (i.e., the same buffer is copied to multiple MRAM banks) is higher than that of parallel CPU-DPU transfers (i.e., different buffers are copied to different MRAM banks) due to higher temporal locality in the CPU cache hierarchy.

Data Transfer: Interleaving & Transposition

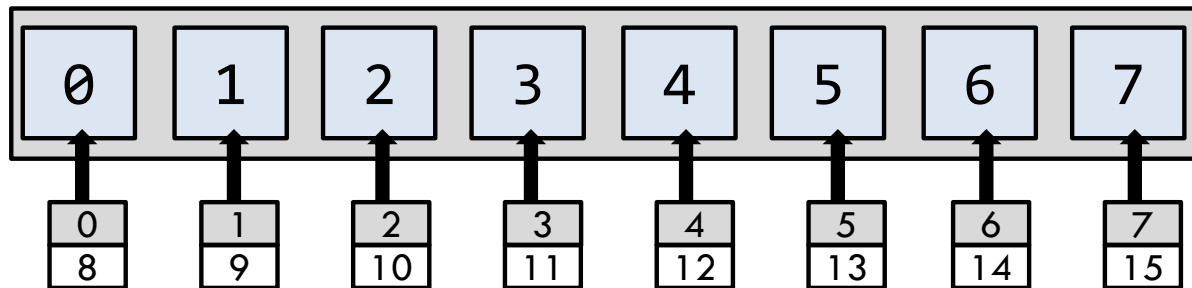
- Transposing library feeds DPUs with the correct data
 - Each 64-bit “horizontal” words are turned into 8 vertical words and fed into 8 different DPU chips



Data Transfer: Interleaving & Transposition

- **Each DRAM chip connects to the DDR4 bus** by an 8-bit interface. 8 chips together form the 64-bit wide bus
- When a 64-byte cache line is committed to memory
 - The data is first interleaved across DRAM chips at byte granularity
 - The first byte is sent to chip 0, the second byte is sent to chip 1, etc.
 - So when the write is done, the **first chip receives bytes 0, 8, 16, ..., 56**

A rank of DPUs (8 DPUs)



**discontinuous
data stream**

The i^{th} bytes of data

DPU Kernel Launch

- `dpu_launch()` launches a kernel on a `dpu_set`
 - `DPU_SYNCHRONOUS` suspends the CPU application until the kernel finishes
 - `DPU_ASYNCHRONOUS` returns the control to the application of the host CPU
 - `dpu_sync` or `dpu_status` to check kernel completion

```
1  printf("Run program on DPU(s) \n");  
2  // Run DPU kernel  
3  DPU_ASSERT(dpu_launch(dpu_set, DPU_SYNCHRONOUS));  
4
```

- What does the asynchronous execution enable?
 - **Task-level parallelism**: concurrent execution of different kernels on different DPU sets
 - Concurrent **heterogeneous computation** on CPU and DPUs

How to Pass Parameters to the Kernel?

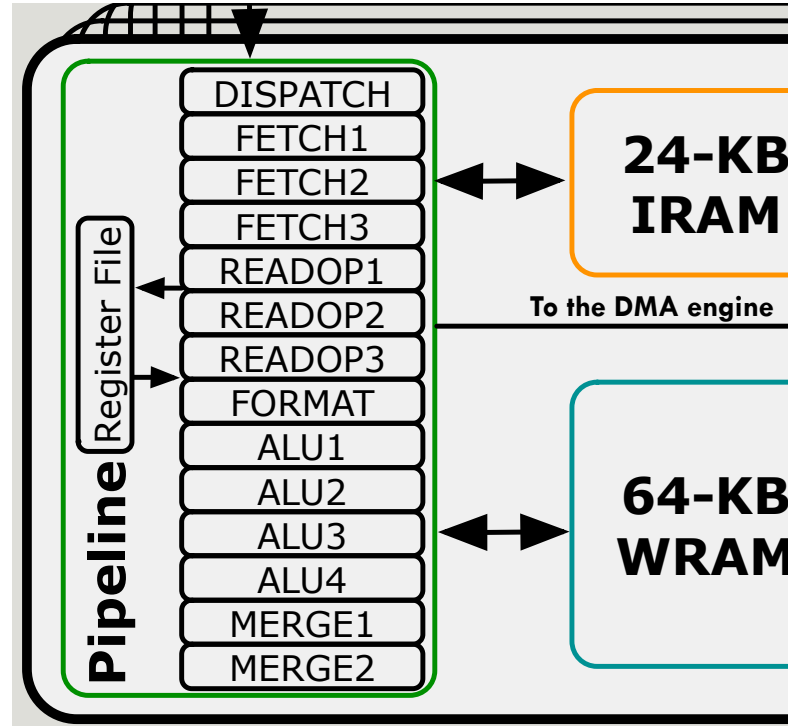
- We can use serial and parallel transfers
- We pass them directly to the scratchpad memory of the DPU
 - Working RAM (WRAM)
- This is useful for **input parameters** and **some results**

```
1 // In DPU WRAM (dpu/task.c)
2 __host dpu_arguments_t DPU_INPUT_ARGUMENTS;
3 __host dpu_results_t DPU_RESULTS[NR_TASKLETS];
4
```

```
1 // Host code (host/app.c)
2 #ifdef SERIAL
3 ▼ DPU_FOREACH (dpu_set, dpu) {
4     DPU_ASSERT(dpu_copy_to(dpu, "DPU_INPUT_ARGUMENTS", 0, (const void *)&input_arguments[i], sizeof(input_arguments[0])));
5     i++;
6 ▲ }
7 #else
8 ▼ DPU_FOREACH(dpu_set, dpu, i) {
9     DPU_ASSERT(dpu_prepare_xfer(dpu, &input_arguments[i]));
10 ▲ }
11 DPU_ASSERT(dpu_push_xfer(dpu_set, DPU_XFER_TO_DPU, "DPU_INPUT_ARGUMENTS", 0, sizeof(input_arguments[0]), DPU_XFER_DEFAULT));
12 #endif
13
```

DPU Pipeline

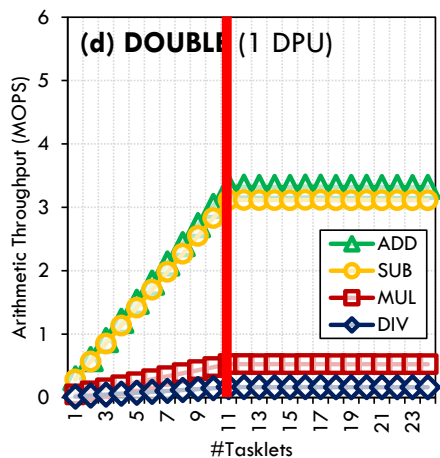
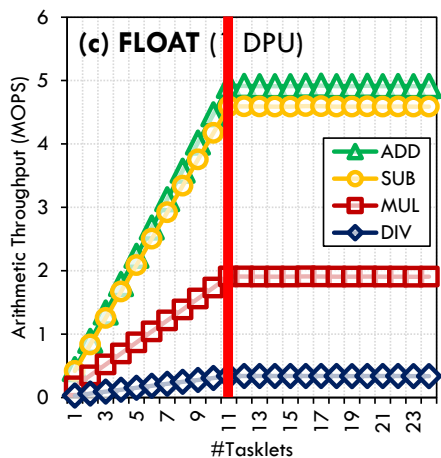
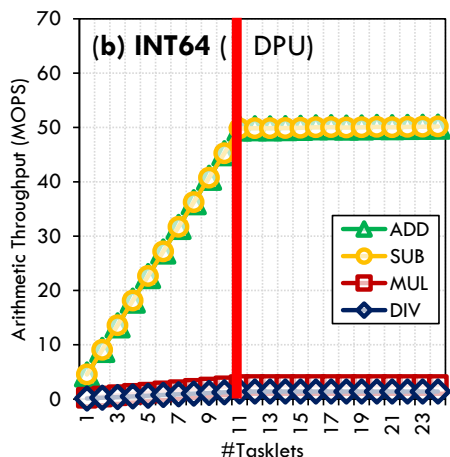
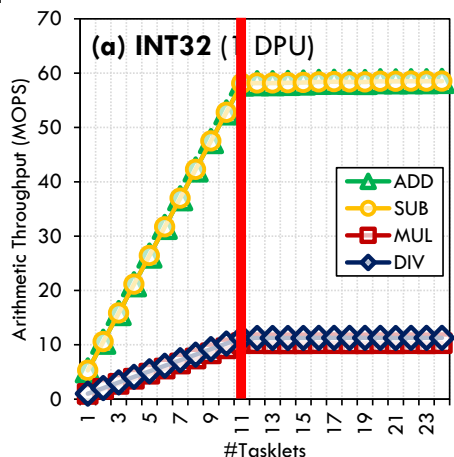
- In-order pipeline
 - Up to 350 MHz
- Fine-grain multithreaded
 - 24 hardware threads
- 14 pipeline stages
 - DISPATCH: Thread selection
 - FETCH: Instruction fetch
 - READOP: Register file
 - FORMAT: Operand formatting
 - ALU: Operation and WRAM
 - MERGE: Result formatting



Arithmetic Throughput: Microbenchmark

- Goal
 - Measure the maximum arithmetic throughput for different datatypes and operations
- Microbenchmark
 - We stream over an array in WRAM and perform read-modify-write operations
 - Experiments on one DPU
 - We vary the number of tasklets from 1 to 24
 - Arithmetic operations: add, subtract, multiply, divide
 - Datatypes: int32, int64, float, double
- We measure cycles with an accurate cycle counter that the SDK provides
 - We include WRAM accesses (including address calculation) and arithmetic operation

Arithmetic Throughput: 11 Tasklets

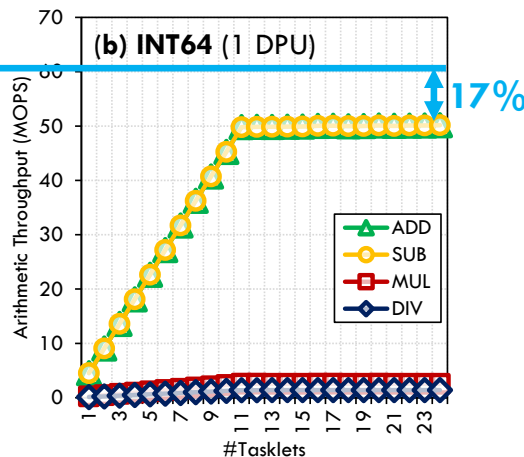
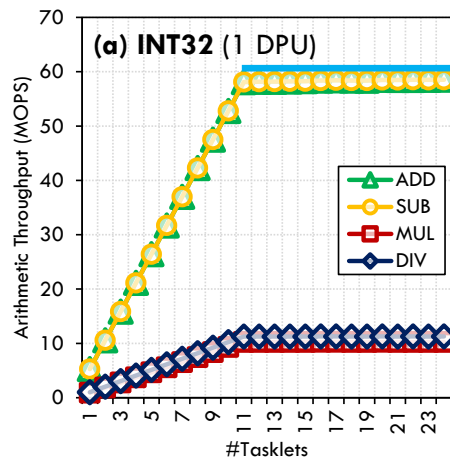


KEY OBSERVATION 1

The arithmetic throughput of a DRAM Processing Unit saturates at 11 or more tasklets.

This observation is consistent for different datatypes (INT32, INT64, UINT32, UINT64, FLOAT, DOUBLE) and operations (ADD, SUB, MUL, DIV).

Arithmetic Throughput: ADD/SUB



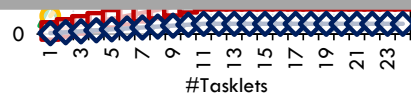
INT32 ADD/SUB are
17% faster than
INT64 ADD/SUB

Can we explain the peak throughput?

Peak throughput at 11 tasklets.

One instruction retires every cycle when the pipeline is full

$$\text{Arithmetic Throughput (in OPS)} = \frac{\text{frequency}_{\text{DPU}}}{\# \text{instructions}}$$



Arithmetic Throughput: #Instructions

- Com

```
1  #define BLOCK_SIZE 1024
```

```
2  typedef int T;
```

```
3  void Benchmark_32bits(T *cache_A, T scalar) {
```

```
4      for (int i = 0; i < BLOCK_SIZE / sizeof(T); i++){
```

```
5          // WRAM READ
```

```
6          T temp = cache_A[i];
```

```
7          temp += scalar; // ADD
```

```
8          // WRAM WRITE
```

```
9          cache_A[i] = temp;
```

```
10     }
```

```
11 }
```

```
12 typedef long T_long;
```

```
13 void Benchmark_64bits(T_long *cache_A, T_long scalar) {
```

```
14     for (int i = 0; i < BLOCK_SIZE / sizeof(T_long); i++){
```

```
15         // WRAM READ
```

```
16         T_long temp = cache_A[i];
```

```
17         temp += scalar; // ADD
```

```
18     }
```

```
19 }
```

```
20
```

```
21
```

```
22
```

```
23
```

```
24
```

```
25
```

```
26
```

```
27
```

A ▾ ☐ 11010 ☐ ./a.out ☒ .LX0: ☒ .text ☒ // ☐ v

```
1 Benchmark_32bits:
```

```
2     move r2, 0
```

```
3 .LBB0_1:
```

```
4     lsl_add r3, r0, r2, 2
```

```
5     lw r4, r3, 0
```

```
6     add r4, r4, r1
```

```
7     sw r3, 0, r4
```

```
8     add r2, r2, 1
```

```
9     jneq r2, 256, .LBB0_1
```

```
10    jump r23
```

```
11 Benchmark_64bits:
```

```
12     move r1, 0
```

```
13 .LBB1_1:
```

```
14     lsl_add r4, r0, r1, 3
```

```
15     ld d6, r4, 0
```

```
16     add r7, r7, r3
```

```
17     addc r6, r6, r2
```

```
18     sd r4, 0, d6
```

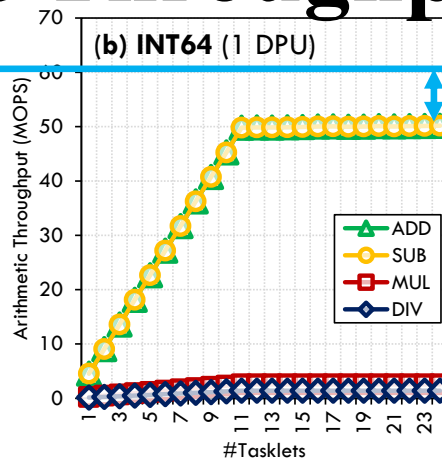
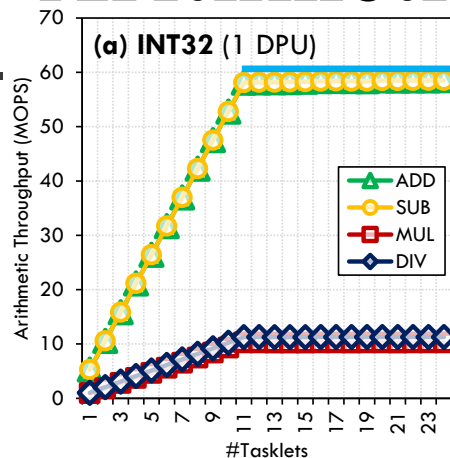
```
19     add r1, r1, 1
```

```
20     jneq r1, 128, .LBB1_1
```

```
21    jump r23
```

6 instructions in the 32-bit ADD/SUB microbenchmark
7 instructions in the 64-bit ADD/SUB microbenchmark

Arithmetic Throughput: ADD/SUB



INT32 ADD/SUB are
17% faster than
INT64 ADD/SUB

Can we explain the peak throughput?

Peak throughput at 11 tasklets.

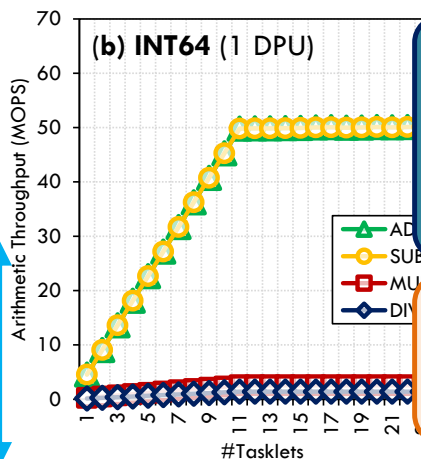
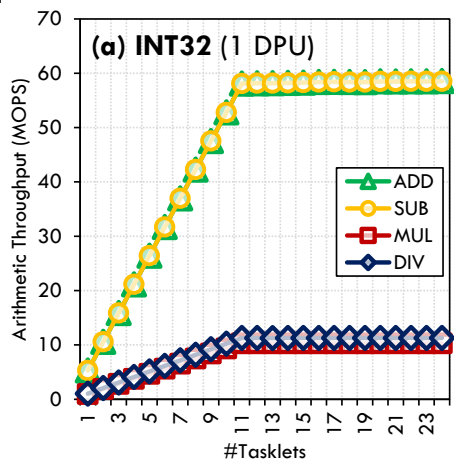
One instruction retires every cycle when the pipeline is full

$$\text{Arithmetic Throughput (in OPS)} = \frac{\text{frequency}_{DPU}}{\#instructions}$$

64-bit ADD/SUB: 7 instructions $\rightarrow$ 50.00 MOPS

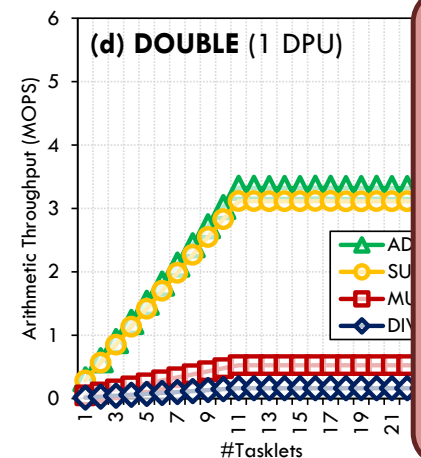
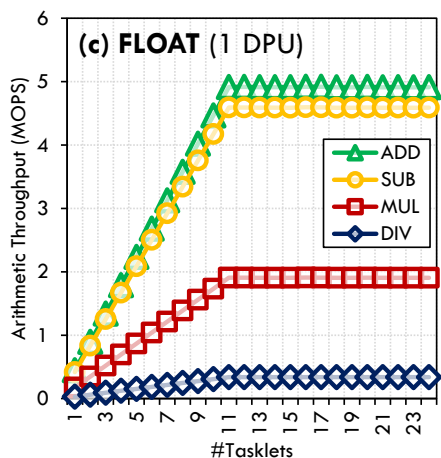
at $\text{frequency}_{DPU} = 350 \text{ MHz}$

Arithmetic Throughput: MUL/DIV



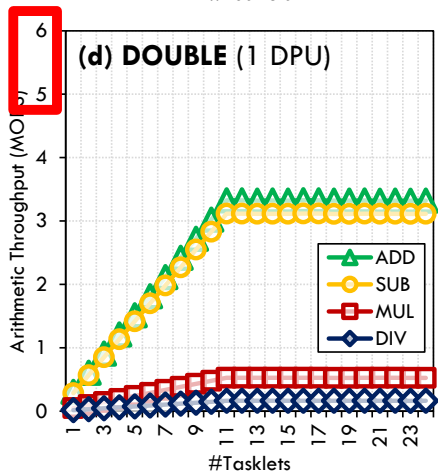
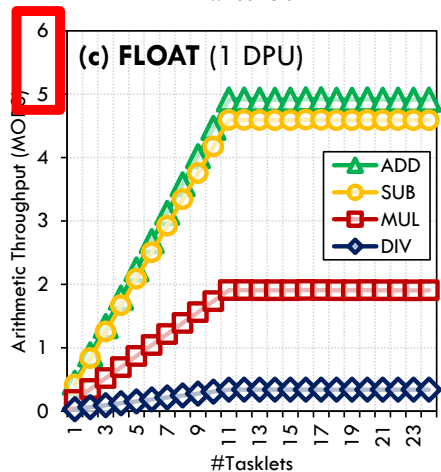
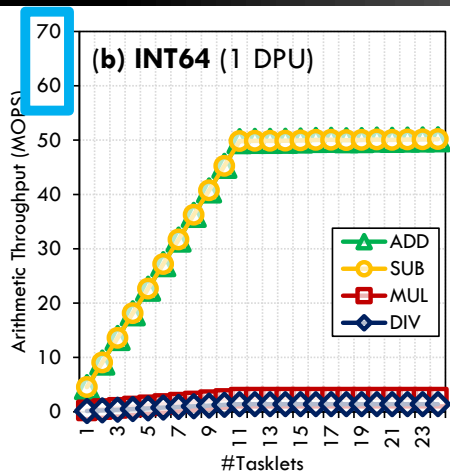
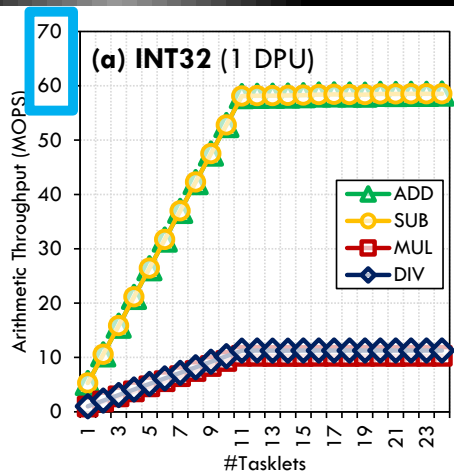
Huge throughput difference between ADD/SUB and MUL/DIV

DPU's do *not* have a 32-bit multiplier



MUL/DIV implementation is based on an instruction that performs *bit shifting and addition* in 1 cycle (MUL/DIV take a maximum of 32 instructions)

Arithmetic Throughput: Native Support

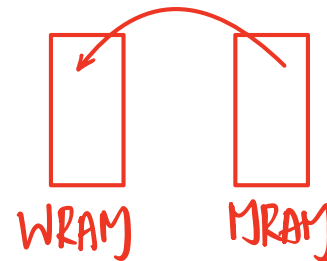


KEY OBSERVATION 2

- DPUs provide **native hardware support** for 32- and 64-bit integer addition and subtraction, leading to high throughput for these operations.
- DPUs do **not** natively support 32- and 64-bit multiplication and division, and floating point operations. These operations are **emulated by the UPMEM runtime library**, leading to much lower throughput.

WRAM Bandwidth: Microbenchmark

- Goal
 - Measure the **WRAM bandwidth** for the STREAM benchmark
- Microbenchmark
 - We implement the four versions of STREAM: **COPY, ADD, SCALE, and TRIAD**
 - The operations performed in ADD, SCALE, and TRIAD are **addition, multiplication, and addition+multiplication**, respectively
 - We vary the number of tasklets from 1 to 16
 - We show results for 1 DPU
- We do *not* include accesses to MRAM



STREAM Benchmark in WRAM

// COPY

```
for(int i = 0; i < SIZE; i++){  
    bufferB[i] = bufferA[i];  
}
```

8 bytes read, 8 bytes written,
no arithmetic operations

// ADD

```
for(int i = 0; i < SIZE; i++){  
    bufferC[i] = bufferA[i] + bufferB[i];  
}
```

16 bytes read, 8 bytes written,
ADD

// SCALE

```
for(int i = 0; i < SIZE; i++){  
    bufferB[i] = scalar * bufferA[i];  
}
```

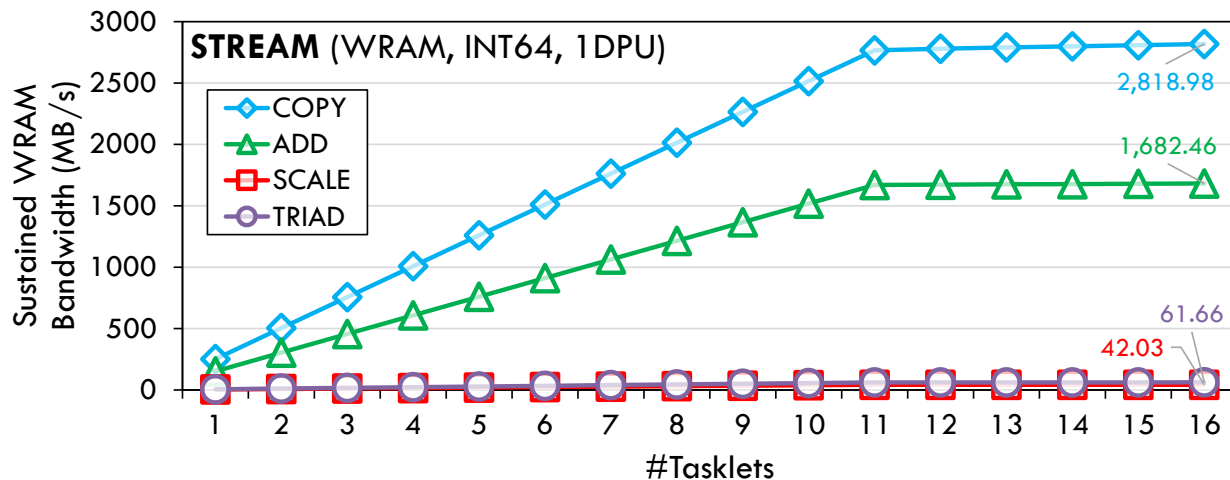
8 bytes read, 8 bytes written,
MUL

// TRIAD

```
for(int i = 0; i < SIZE; i++){  
    bufferC[i] = bufferA[i] + scalar * bufferB[i];  
}
```

16 bytes read, 8 bytes written,
MUL, ADD

WRAM Bandwidth: STREAM

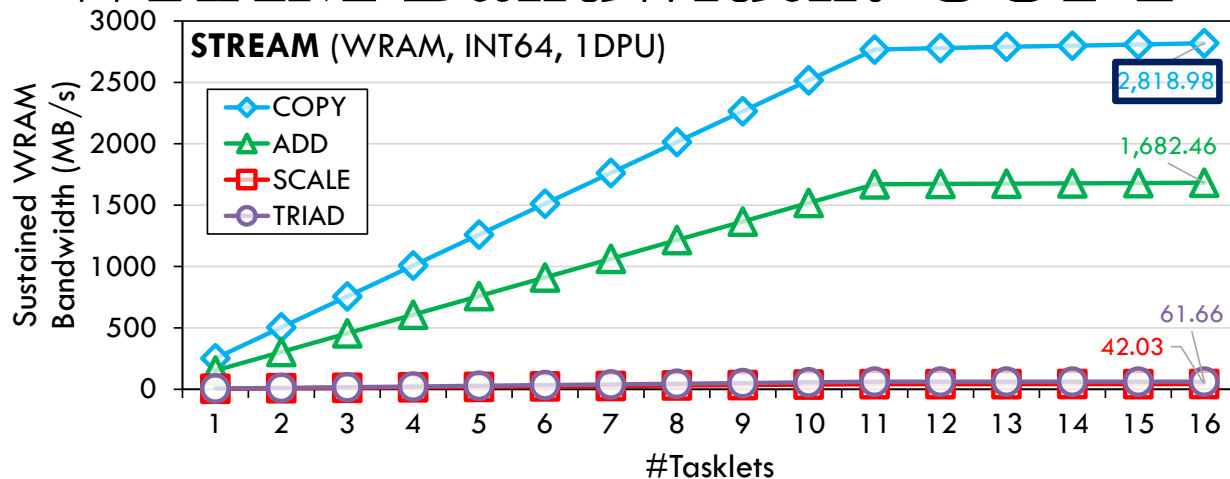


How can we estimate the bandwidth?

Assuming that the pipeline is full, and *Bytes* is the number of bytes read and written:

$$WRAM\ Bandwidth\ \left(in\ \frac{B}{S}\right) = \frac{Bytes \times frequency_{DPU}}{\#instructions}$$

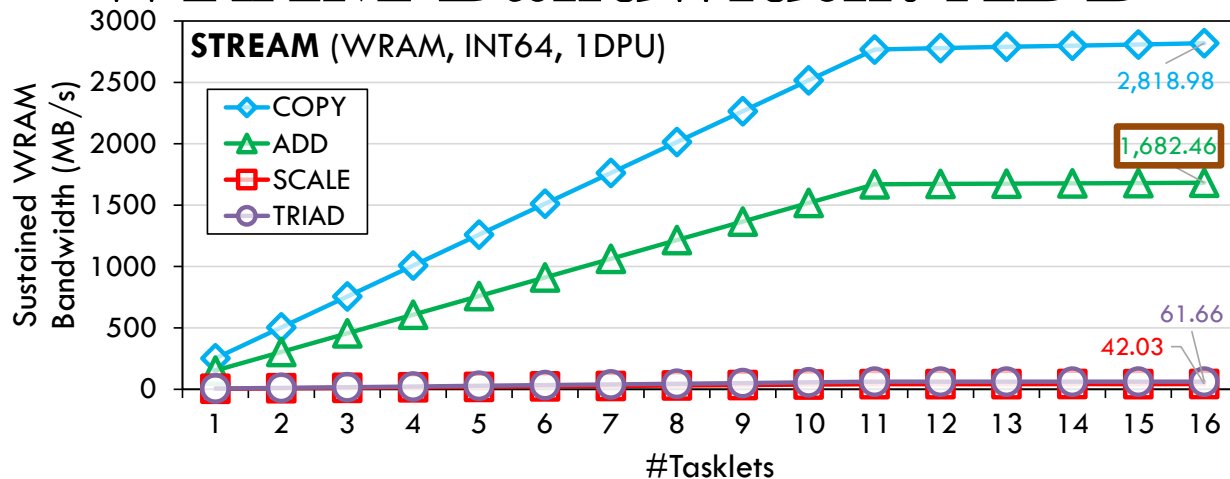
WRAM Bandwidth: COPY



COPY executes 2 instructions (WRAM load and store).
With 11 tasklets, 11×16 bytes in 22 cycles:

$$WRAM \text{ Bandwidth } \left(in \frac{B}{S} \right) = 2,800 \frac{MB}{s} \text{ at } 350 \text{ MHz}$$

WRAM Bandwidth: ADD



$$WRAM\ Bandwidth\ \left(in\ \frac{B}{S}\right) = \frac{Bytes \times frequency_{DPU}}{\#instructions}$$

ADD executes **5 instructions** (2 ld, add, addc, sd).

With 11 tasklets, **11 × 24 bytes** in **55 cycles**:

$$WRAM\ Bandwidth\ \left(in\ \frac{B}{S}\right) = 1,680 \frac{MB}{s} \text{ at } 350\ MHz$$

WRAM Bandwidth: Access Patterns

- All 8-byte **WRAM loads and stores take one cycle** when the DPU pipeline is full

KEY OBSERVATION 3

The sustained bandwidth provided by the DPU's internal Working memory (WRAM) is **independent of the memory access pattern** (either streaming, strided, or random access pattern).

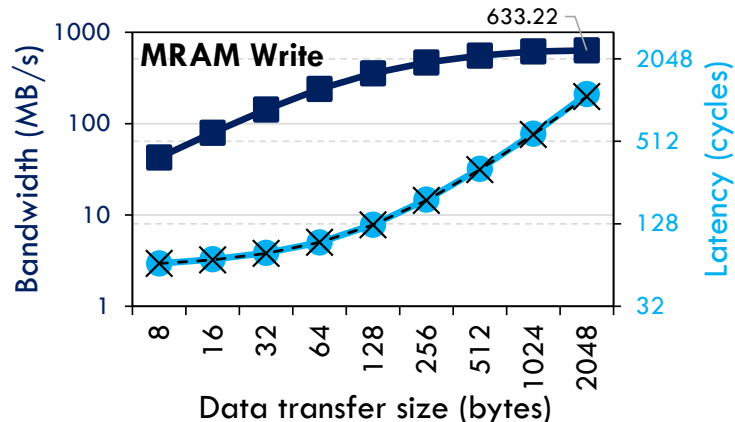
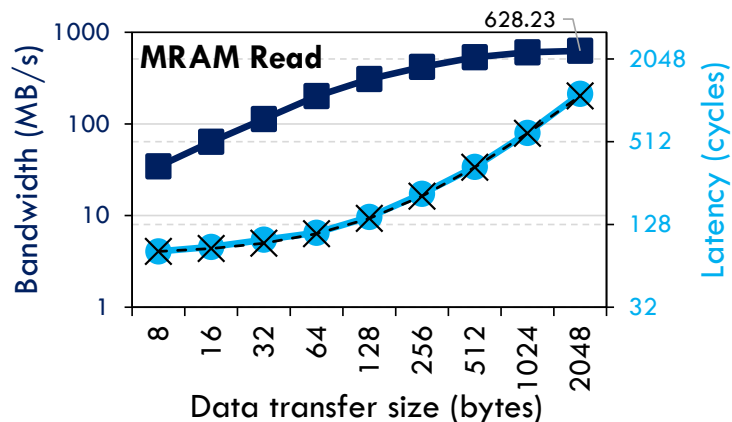
All 8-byte WRAM loads and stores take one cycle, when the DPU's pipeline is full (i.e., with 11 or more tasklets).

- Microbenchmark: `c[a[i]]=b[a[i]];`
 - Unit-stride: `a[i]=a[i-1]+1;`
 - Strided: `a[i]=a[i-1]+stride;`
 - Random: `a[i]=rand();`

MRAM Bandwidth

- Goal
 - Measure **MRAM bandwidth** for different access patterns
- Microbenchmarks
 - **Latency of a single DMA transfer** for different transfer sizes
 - `mram_read();` // MRAM-WRAM DMA transfer
 - `mram_write();` // WRAM-MRAM DMA transfer
 - **STREAM** benchmark
 - COPY, COPY-DMA
 - ADD, SCALE, TRIAD
 - **Strided** access pattern
 - Coarse-grain strided access
 - Fine-grain strided access
 - **Random** access pattern (GUPS)
- We do include accesses to MRAM

MRAM Read and Write Latency (I)



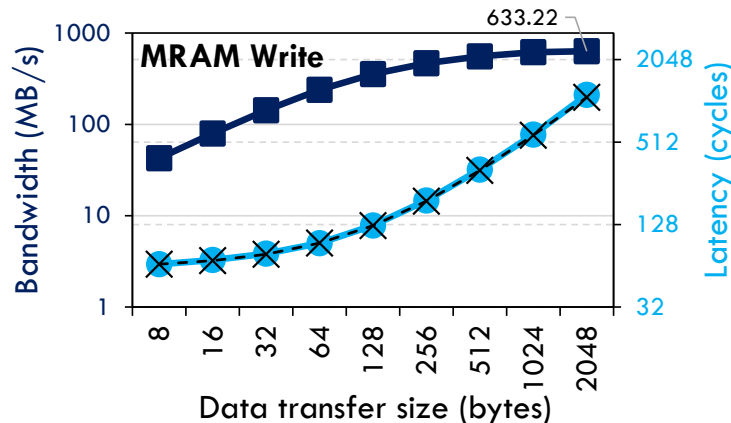
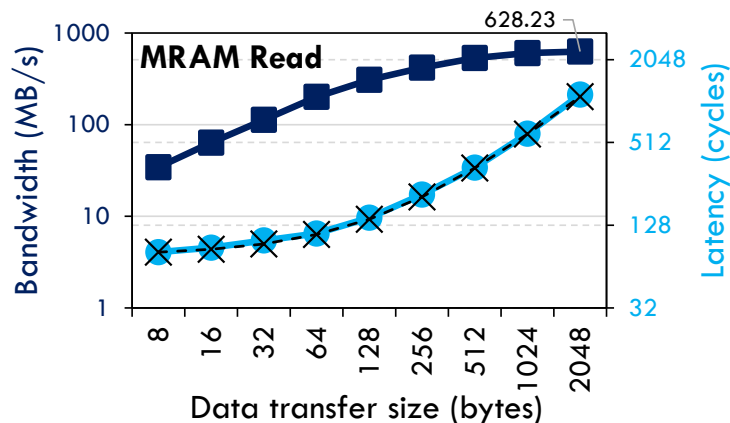
$$MRAM \text{ Bandwidth } \left(\text{in } \frac{B}{S} \right) = \frac{\text{size} \times \text{frequency}_{DPU}}{MRAM \text{ Latency}}$$

We can model the MRAM latency with a linear expression

$$MRAM \text{ Latency (in cycles)} = \alpha + \beta \times \text{size}$$

In our measurements, β equals 0.5 cycles/byte.
Theoretical maximum MRAM bandwidth = 700 MB/s at 350 MHz

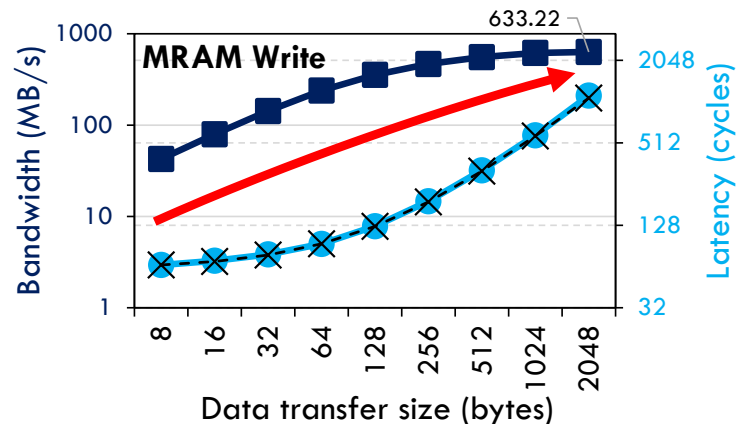
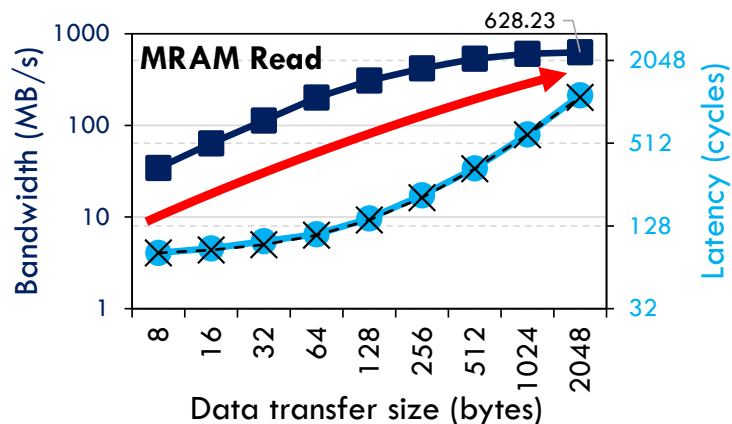
MRAM Read and Write Latency (II)



KEY OBSERVATION 4

- The DPU's Main memory (MRAM) bank access latency increases **linearly** with the transfer size.
- The maximum theoretical MRAM bandwidth is 2 bytes per cycle.

MRAM Read and Write Latency (III)



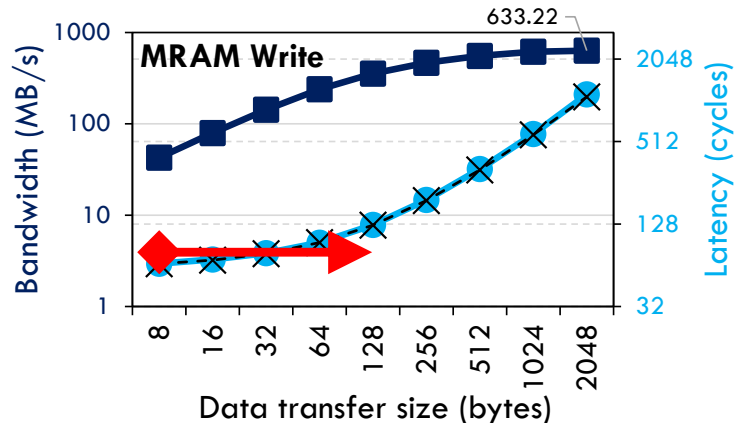
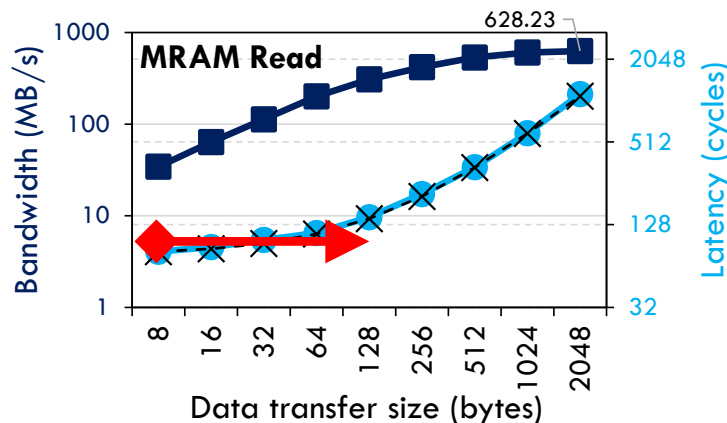
Read and write accesses to MRAM are symmetric

The sustained MRAM bandwidth increases with data transfer size

PROGRAMMING RECOMMENDATION 1

For data movement between the DPU's MRAM bank and the WRAM, **use large DMA transfer sizes when all the accessed data is going to be used.**

MRAM Read and Write Latency (IV)



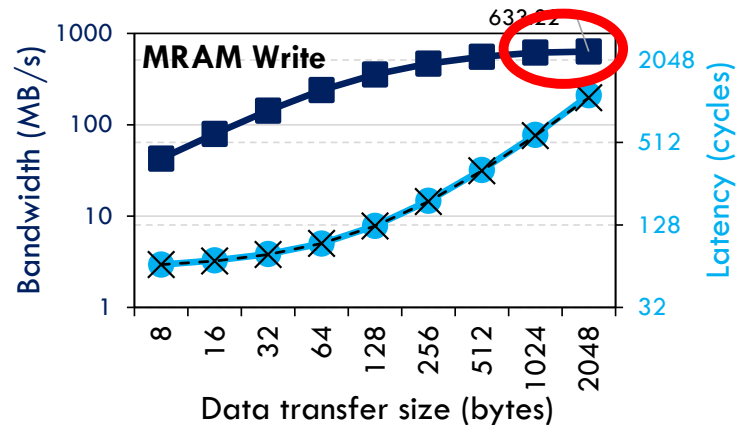
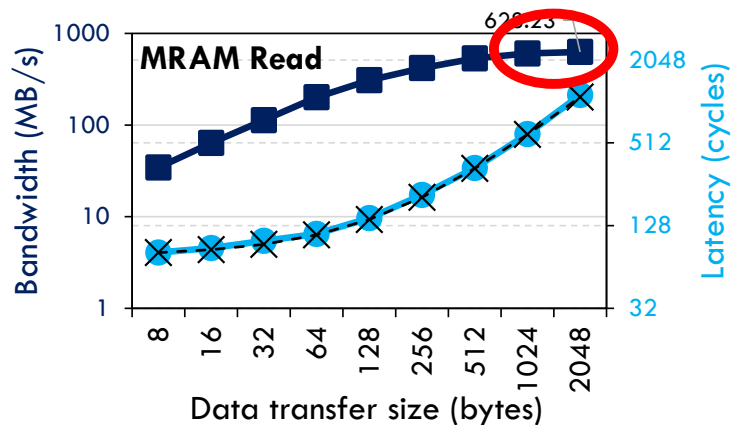
MRAM latency changes slowly between 8 and 128 bytes

For small transfers, the fixed cost (α) dominates the variable cost ($\beta \times size$)

PROGRAMMING RECOMMENDATION 2

For small transfers between the MRAM bank and the WRAM, **fetch more bytes than necessary within a 128-byte limit**. Doing so increases the likelihood of finding data in WRAM for later accesses (i.e., the program can check whether the desired data is in WRAM before issuing a new MRAM access).

MRAM Read and Write Latency (V)



2,048-byte transfers are only 4% faster than 1,024-byte transfers

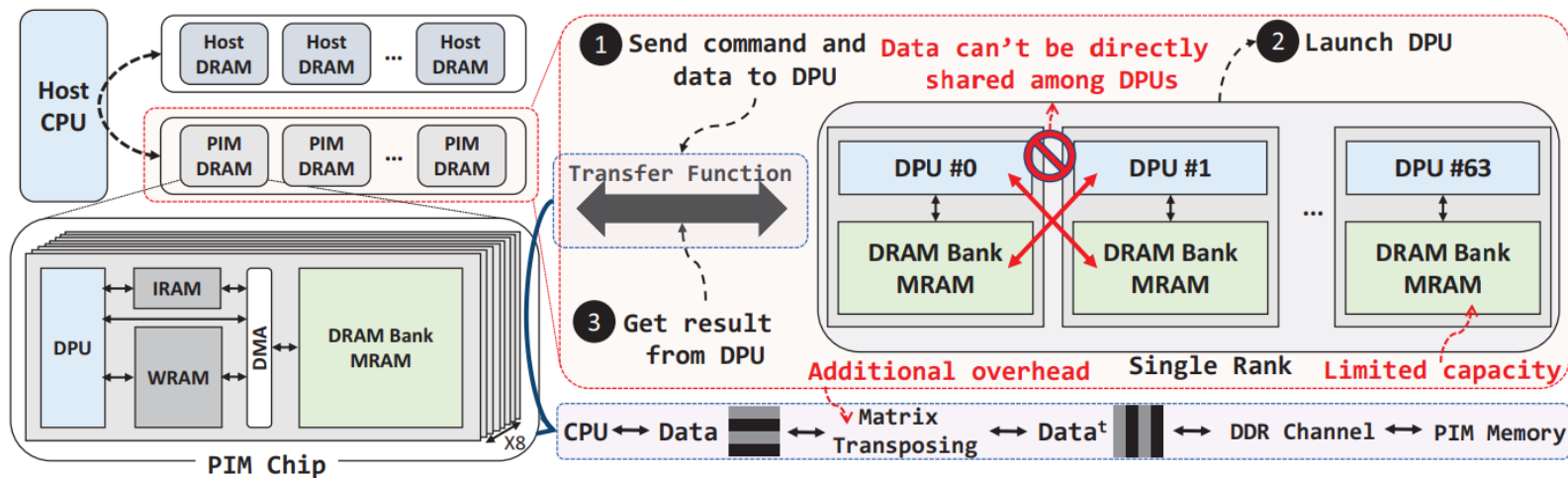
Larger transfers require more WRAM, which may limit the number of tasklets

PROGRAMMING RECOMMENDATION 3

Choose the data transfer size between the MRAM bank and the WRAM based on the program's WRAM usage, as it imposes a tradeoff between the sustained MRAM bandwidth and the number of tasklets that can run in the DPU (which is dictated by the limited WRAM capacity).

Summary: How DPU Works?

- How to use DPU
 1. Send command and input data to DPU by transfer API provided by UPMEM SDK
 2. Launch DPU which means DPU execution
 3. Get result from DPU by transfer API



Constraints and limitation of DPU

- Several rules need to be followed when using DPU
 - CPU access MRAM only by transfer APIs
 - Granularity of communication: Rank
 - Data transfer and DPU execution can't work concurrently
- ★ No data sharing among DPUs

